

PLATAFORMA INTELIGENTE PARA
RECRUITING ÓPTIMO DE PERSONAL
FREELANCE

Contenido

PLATAFORMA INTELIGENTE PARA RECRUITING ÓPTIMO DE PERSONAL FREELANCE

Contenido

I. Introducción

1.1 Objetivos

Objetivo general

Objetivos empresariales

Objetivos tecnológicos

Objetivos específicos

1.2 Audiencia

1.3 Justificación

II. Marco conceptual

2.1 Elementos clave

2.2 Ventajas

2.3 Desafíos

2.4 Ciclo de vida

2.5 Descripción del producto o tecnología

Funcionalidades

Beneficios

Características

III. Customer Journey

3.1 Descubrimiento

3.2 Consideración

3.3 Decisión

3.4 Secuencia de actividades (experiencia de usuario)

3.5 Fidelización

3.6 Promoción

3.7 Diagramación

3.8 Wireframe

3.9 UI

3.10 Arquitectura del Proyecto

Componentes

Despliegue

IV. Metodología de desarrollo

4.1 Introducción

4.2 Implementación de la Metodología

V. Desarrollo de la tecnología

5.1 Introducción

5.2 Diseño e implementación

Frontend

Backend

1. Lógica del servidor Parse Server y Express

[Características principales de Parse Server:](#)

[Beneficios de usar Parse:](#)

[2. Bases de datos y almacenamiento en el backend](#)

[Características principales de MongoDB](#)

[Beneficios de MongoDB](#)

[Base de datos](#)

[Modelo de Datos](#)

[1. Diagrama de Datos y Esquema de la base de datos](#)

[2. Datos clave-valor y Optimización Redis](#)

[3. Políticas de Seguridad](#)

[Descripción de la arquitectura del backend](#)

[Inteligencia Artificial](#)

[Origen y Naturaleza de los Datos](#)

[Datos Scrapeados](#)

[Datos Simulados](#)

[Proceso de Limpieza y Preprocesamiento de Datos](#)

[Código](#)

[Elección del Algoritmo de Machine Learning](#)

[Implementación del algoritmo](#)

[Pruebas y Ajustes Fines](#)

I Introducción

En el dinámico panorama laboral actual, la demanda de talento freelance ha experimentado un crecimiento exponencial. Las empresas buscan cada vez más profesionales independientes altamente calificados para complementar sus equipos y abordar proyectos específicos. Sin embargo, el proceso de reclutamiento tradicional puede resultar engorroso y poco eficiente.

Para revolucionar esta dinámica y ofrecer una solución innovadora, OrionTech ha desarrollado una plataforma inteligente de reclutamiento que aprovecha el poder de la inteligencia artificial y el análisis de datos. Esta herramienta pionera está diseñada para evaluar de manera precisa y objetiva las habilidades de los candidatos freelance, optimizando así el proceso de selección y garantizando que las empresas encuentren el talento ideal para cada proyecto.

1.1 Objetivos

Objetivo general

Desarrollar una plataforma inteligente de reclutamiento que evalúe tanto habilidades específicas como generales de cada candidato, utilizando metodologías Data-Driven para calificar y clasificar automáticamente a los candidatos en función de sus resultados en pruebas diseñadas por especialistas y ejecutadas por inteligencia artificial. Esto permite al equipo de recursos humanos basarse en datos confiables para tomar decisiones importantes sobre la selección de personal freelance.

Objetivos empresariales

Revolucionar la forma en que las empresas encuentran y contratan talento independiente. El objetivo es reducir significativamente el tiempo y costo asociado al proceso de reclutamiento, mejorar la calidad de los candidatos y aumentar la satisfacción del cliente. Busca conectar a empresas con los mejores freelancers del mercado, garantizando una experiencia de contratación eficiente, segura y transparente.

Objetivo científico-tecnológico

Emplear tecnologías avanzadas como Inteligencia Artificial (IA), Machine Learning y Análisis de Datos para desarrollar un sistema de recomendación de candidatos óptimos. Utilizará algoritmos de matching avanzados para conectar a empresas con freelancers que mejor se ajusten a sus necesidades específicas. Además, implementaremos herramientas de evaluación y seguimiento para garantizar la calidad y productividad de los proyectos.

Objetivos Específicos

- Desarrollar un algoritmo de evaluación que analice y puntúe las habilidades específicas de los candidatos en áreas como programación, marketing digital y otras más relevantes para el proyecto.

- Implementar una interfaz de usuario intuitiva y amigable que permita a los empleadores crear pruebas personalizadas para evaluar a los candidatos.
- Integrar tecnologías de inteligencia artificial para ejecutar las pruebas diseñadas y recopilar datos sobre el rendimiento de los candidatos de manera automatizada.
- Diseñar un sistema de calificación automática que asigne puntajes objetivos a cada candidato en función de su desempeño en las pruebas y otros criterios definidos.
- Establecer una clasificación automática de los candidatos en función de los puntajes obtenidos en las pruebas, priorizando aquellos con las calificaciones más altas.
- Garantizar la seguridad y la privacidad de los datos de los candidatos durante todo el proceso de evaluación y clasificación.
- Realizar pruebas exhaustivas de la plataforma para identificar y corregir posibles errores o deficiencias en su funcionamiento.
- Ofrecer soporte técnico y capacitación adecuada al equipo de recursos humanos para utilizar eficazmente la plataforma y tomar decisiones informadas basadas en los datos proporcionados

1.2 Audiencia

Este proyecto estaría dirigido a empresas u organizaciones que buscan reclutar personal freelance de manera eficiente y basada en datos. Estas empresas pueden pertenecer a diversos sectores, como tecnología, marketing, diseño, consultoría, entre otros, y pueden tener una amplia gama de necesidades de contratación de talento independiente para proyectos específicos. La plataforma inteligente de reclutamiento podría ser especialmente útil para equipos de recursos humanos que deseen optimizar el proceso de selección de personal freelance, asegurando la calidad y la idoneidad de los candidatos para las tareas requeridas.

1.3 Justificación

La plataforma fue creada para resolver el desafío crítico de encontrar y contratar talento freelance de manera eficiente, confiable y justa. Actualmente, los procesos de selección suelen ser largos, subjetivos y costosos, lo que limita el acceso a los mejores talentos y retrasa proyectos importantes. Este sistema inteligente busca revolucionar el reclutamiento freelance, garantizando que las decisiones de contratación se basen en datos objetivos, aumentando la productividad de las empresas y ofreciendo a los freelancers oportunidades transparentes y equitativas para destacar sus habilidades.

II. Marco conceptual

La plataforma inteligente de reclutamiento óptimo de personal freelance está concebida bajo los principios de la metodología Lean Development y tiene como objetivo revolucionar la manera en que las empresas encuentran y contratan talento independiente.

La plataforma propuesta tiene como objetivo la implementación de tecnologías avanzadas de inteligencia artificial y análisis de datos, se busca optimizar el proceso de selección, garantizando la identificación de los candidatos más adecuados para cada proyecto.

2.1 Elementos clave

El proyecto para la plataforma inteligente de reclutamiento combina los siguientes elementos de la metodología Lean Development:

- **Enfoque en el Progreso:** Se prioriza la entrega continua de valor al usuario mediante ciclos de desarrollo ágiles que permiten ajustes rápidos basados en retroalimentación.
- **Eliminación de desperdicio:** Se identifican y eliminan actividades que no aportan valor al producto final, optimizando así los recursos y el tiempo de desarrollo.
- **Mejora Continua:** Se promueve una cultura de aprendizaje constante, donde se realizan retrospectivas regulares para identificar áreas de mejora y aplicar lecciones aprendidas.
- **Visualización del Flujo de Trabajo:** Se utilizan archivos de control para visualizar el progreso del desarrollo, lo que facilita la identificación de cuellos de botella y permite ajustes dinámicos.
- **Testing Continuo y Refactorización:** Se implementan prácticas como pruebas automatizadas y revisiones de código para asegurar la calidad del software a lo largo del ciclo de vida del desarrollo.

2.2 Ventajas

La implementación de una plataforma que combine algoritmos de inteligencia artificial, para conectar a empresas con freelancers que mejor se ajusten a sus necesidades específicas presenta una serie de ventajas significativas tanto para las empresas como los freelancers:

Para las Empresas:

- **Ahorro de tiempo:** Automatización de tareas repetitivas como la revisión de currículums y la programación de entrevistas, lo que permite a los reclutadores centrarse en tareas más estratégicas.
- **Mayor precisión en la selección:** Identificación de candidatos altamente cualificados y que se ajustan perfectamente al perfil del puesto, reduciendo el riesgo de contrataciones erróneas.
- **Reducción de costos:** Optimización de los recursos destinados al reclutamiento, al reducir el tiempo dedicado a la búsqueda y selección de candidatos.

- **Mejora de la experiencia del candidato:** Un proceso de selección más rápido y eficiente genera una mejor percepción de la empresa entre los candidatos.
- **Acceso a un pool de talento global:** La plataforma permite acceder a una amplia base de datos de freelancers de todo el mundo, facilitando la búsqueda de perfiles especializados.
- **Toma de decisiones basada en datos:** La plataforma proporciona información valiosa sobre los candidatos, lo que permite a las empresas tomar decisiones más informadas y estratégicas.

Para los Freelancers:

- **Mayor visibilidad:** Los freelancers pueden dar a conocer sus habilidades y experiencia a un amplio público de empresas.
- **Acceso a oportunidades laborales:** La plataforma les permite encontrar proyectos que se ajusten a sus intereses y experiencia.
- **Proceso de selección más justo y transparente:** La evaluación de las habilidades se basa en criterios objetivos y transparentes.
- **Gestión eficiente de sus proyectos:** La plataforma puede ayudar a los freelancers a gestionar sus proyectos y a mantener una comunicación fluida con los clientes.

2.3 Desafíos

A pesar de sus numerosas ventajas, el proyecto enfrenta ciertos desafíos:

- **Desarrollo tecnológico:** El desarrollo de una plataforma de reclutamiento inteligente plantea desafíos técnicos significativos, como la creación de algoritmos sofisticados de machine learning capaces de evaluar de forma precisa las habilidades de los candidatos, la integración de diversas fuentes de datos y el diseño de una arquitectura escalable que pueda soportar un crecimiento exponencial de usuarios y datos.
- **Aceptación del mercado:** Convencer a las empresas a adoptar una herramienta de reclutamiento basada en inteligencia artificial representa un desafío considerable. La desconfianza en la capacidad de las máquinas para tomar decisiones de reclutamiento tan precisas como los humanos, la necesidad de cambiar paradigmas en los procesos de selección y la competencia de otras soluciones en el mercado son obstáculos a superar.
- **Regulación y normativas:** El cumplimiento de las regulaciones de protección de datos, es fundamental para garantizar la privacidad y seguridad de los datos de los candidatos. Además, es necesario diseñar algoritmos que eviten cualquier tipo de discriminación y garantizar la transparencia de los procesos de evaluación.
- **Mantenimiento de la calidad:** Asegurar que los estándares Lean se mantengan durante todo el ciclo de vida del producto es crucial para su éxito. Mantener la relevancia de la plataforma en un mercado laboral en constante evolución es un desafío continuo. Esto implica actualizar los algoritmos para reflejar las nuevas habilidades requeridas, garantizar una experiencia de usuario óptima y mejorar continuamente la plataforma en función de los comentarios de los usuarios.

2.4 Ciclo de vida

El ciclo de vida del proyecto para la plataforma inteligente de reclutamiento se puede dividir en varias fases:

1. **Investigación y Desarrollo:** Realizar un análisis del mercado de reclutamiento freelance para identificar las necesidades específicas de empresas y freelancers.
Desarrollo Tecnológico: Se desarrollará la plataforma utilizando tecnologías de vanguardia de inteligencia artificial, para evaluar las habilidades de los candidatos, recomendar perfiles adecuados y optimizar el proceso de selección. Además, se diseñará una interfaz intuitiva y fácil de usar tanto para empresas como para freelancers.
2. **Pruebas y Validación:** Evaluación del producto mediante pruebas beta, asegurando que cumpla con los estándares establecidos por Lean Development e ISO.
3. **Lanzamiento al mercado:** Introducción oficial de la plataforma inteligente para reclutamiento al público objetivo, acompañado por campañas promocionales.
4. **Monitoreo y Mejora Continua:** Seguimiento del rendimiento de proyecto, recolección de feedback y ajustes necesarios para mejorar la experiencia del usuario.

2.5 Descripción del producto o tecnología

La plataforma inteligente de reclutamiento óptimo de persona freelance es una solución que emplea tecnologías avanzadas como Inteligencia Artificial (IA), Machine Learning y Análisis de Datos para desarrollar un sistema de recomendación de candidatos óptimos. Permite conectar a empresas con freelancers que mejor se ajusten a sus necesidades específicas. Además, contar con herramientas de evaluación y seguimiento para garantizar la calidad y productividad de los proyectos.

Funcionalidades

Además, nuestra plataforma ofrece una amplia gama de funcionalidades para el reclutamiento, incluyendo:

- Evaluación de Habilidades: Análisis y puntuación de habilidades específicas de los candidatos.
- Creación de Pruebas Personalizadas: Interfaz intuitiva para diseñar pruebas adaptadas a las necesidades del proyecto.
- Sistema de Recomendaciones: Integración de IA para generar recomendaciones inteligentes.
- Calificación Automática: Asignación de puntajes objetivos a candidatos basados en su desempeño en las pruebas.
- Clasificación Automática: Priorización de candidatos según puntajes obtenidos en pruebas.
- Pruebas exhaustivas: Identificación y corrección de errores para un funcionamiento óptimo.

- Soporte y Capacitación: Asistencia técnica y capacitación para el uso efectivo de la plataforma.

Beneficios

La plataforma inteligente de reclutamiento ofrece beneficios para las empresas empleadoras y los candidatos independientes.

Para las empresas:

- Mejora la eficiencia del proceso de reclutamiento
- Incrementa la precisión en la selección de personal freelance
- Asegura la calidad y la idoneidad de los candidatos
- Reduce el tiempo y los costos asociados con la búsqueda de talento

Para los candidatos:

- Ofrece una oportunidad más justa y equitativa para mostrar sus habilidades y experiencia
- Permite a los candidatos ser evaluados objetivamente en función de su desempeño
- Ayuda a los candidatos a encontrar proyectos que se adapten a sus habilidades e intereses

Características

Nuestra plataforma cuenta con una amplia gama de características que lo hacen único en el mercado. Algunas de estas características incluyen:

- Interfaz de usuario personalizable para crear pruebas.
- Algoritmo de evaluación basado en metodologías Data-Driven.
- Integración de tecnologías de inteligencia artificial para ejecutar las pruebas.
- Sistema de calificación automática que asigna puntajes objetivos a cada candidato.
- Clasificación automática de los candidatos en función de sus puntajes.

III. Customer Journey

Las empresas u organizaciones han comprendido que es necesaria una plataforma inteligente de reclutamiento que evalúe tanto habilidades específicas como generales de cada candidato, utilizando metodologías Data-Driven para calificar y clasificar automáticamente a los candidatos en función de sus resultados en pruebas diseñadas por especialistas y ejecutadas por inteligencia artificial. Sería ideal que la plataforma tenga como objetivo permitir al equipo de recursos humanos basarse en datos confiables para tomar decisiones importantes sobre la selección de personal freelance.

3.1 Descubrimiento

- La adopción de plataformas tecnológicas avanzadas con inteligencia artificial, machine learning y análisis de datos para desarrollar un sistema de recomendación de candidatos óptimos, ha generado con creciente interés entre:
 - **Empresas de todos los tamaños:** Desde startups hasta grandes corporaciones que buscan optimizar sus procesos de contratación y encontrar talento freelance especializado.
 - **Organizaciones sin fines de lucro:** Que necesitan contratar profesionales freelance para proyectos específicos y desean garantizar la calidad de los candidatos.
 - **Agencias de contratación:** Que buscan una herramienta para evaluar y seleccionar de manera eficiente a los candidatos que presentan a sus clientes.
 - **Reclutadores:** Que buscan agilizar sus procesos de selección y encontrar candidatos más calificados.
 - **Gerentes de contratación:** Que necesitan tomar decisiones de contratación basadas en datos objetivos y confiables.
 - **Analistas de talento:** Que se encargan de identificar las necesidades de talento de la organización y de desarrollar estrategias de reclutamiento efectivas.

3.2 Consideración

- El usuario busca más información sobre la plataforma, sus características y beneficios.
- El usuario compara la plataforma con otras que también ofrecen soluciones similares en el mercado.

3.3 Decisión

Los usuarios, tanto empresas como candidatos freelance, optan por decidir utilizar la plataforma inteligente de reclutamiento debido a su capacidad de optimizar significativamente los procesos de selección de personal. Al emplear metodologías basadas

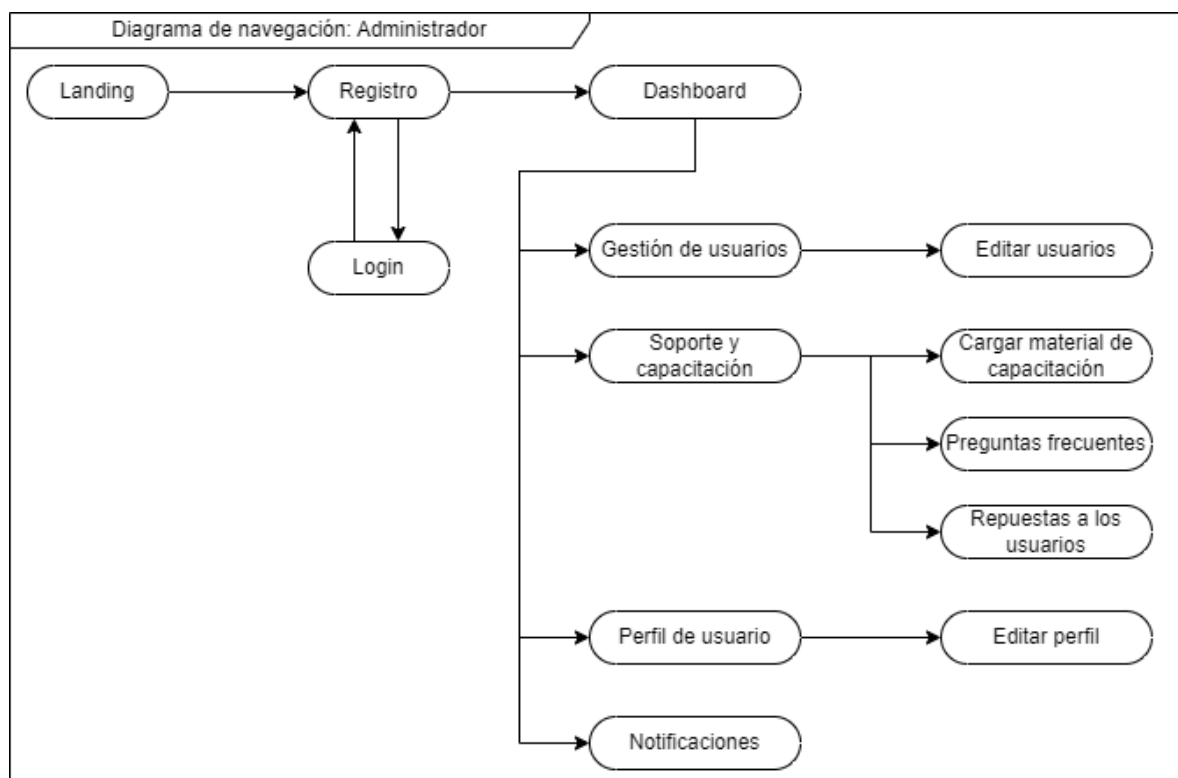
en datos y algoritmos de inteligencia artificial, la plataforma garantiza una evaluación precisa y objetiva de las habilidades de los candidatos, reduciendo así el sesgo humano y el tiempo invertido en la revisión de currículos.

Asimismo, la plataforma ofrece a los equipos de recursos humanos una herramienta confiable para tomar decisiones informadas y estratégicas, basadas en datos sólidos y análisis detallados, lo que a su vez incrementa la probabilidad de encontrar al candidato ideal para cada puesto y mejora la eficiencia en la contratación.

3.4. Secuencia de actividades (experiencia de usuario)

Administrador (Experiencia de usuario)

- Al ingresar al panel de control podrá:
- Gestionar funciones del sistema.
- Manejar datos de usuarios.
- Generar informes.



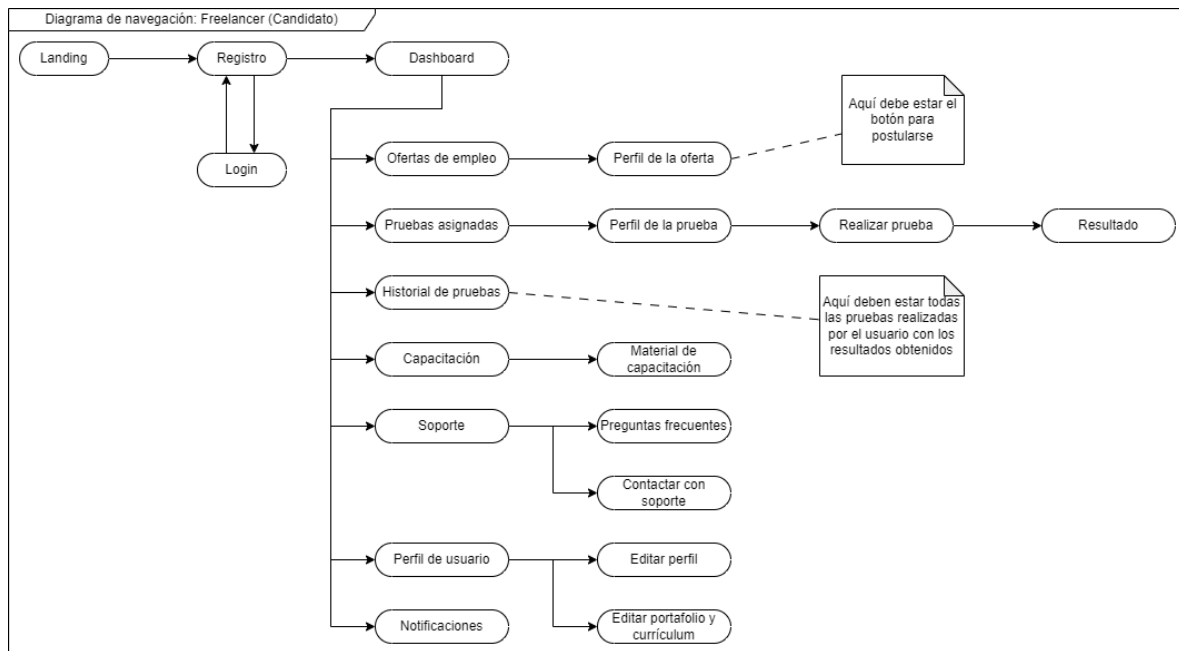
[Ver imagen](#)

Usuario Freelancer (Experiencia de usuario)

- El usuario se registra previamente en la plataforma.
- Inicia sesión con su cuenta.

- Al ingresar al sistema podrá.
 - Acceder a gestión de ofertas de empleo:
 - Podrá añadir, editar o eliminar la oferta de empleo seleccionada.
 - Ingresará datos nuevos de la oferta de empleo.
 - Recibirá notificaciones.
 - Proceso de selección de postulantes:
 - Se postulará a un empleo.
 - Completará la prueba de capacidades.
 - Recibirá la notificación de postulación aceptada o rechazada.
 - Reportes:
 - Ingresará a reportes.
 - Seleccionará el tipo de reportes.
 - Seleccionará generar.
 - Recibirá la notificación de reporte listo.
 - Seleccionará descargar.
 - Tendrá la opción de gestión de usuarios:
 - Ingresará a gestionar datos.
 - Ingresará datos nuevos.
 - Actualizará datos de freelancer (portafolios, resumen curricular y aptitudes).
 - Soporte:
 - Consultará preguntas frecuentes.
 - Seleccionará si desea ponerse con soporte (si o no).
 - Ingresará datos y consulta específica.
 - Recibirá respuesta y solución al problema.
 - Capacitación:
 - Consultará los tutoriales y guías disponibles.
 - Seleccionará la deseada.

- Realizará la capacitación.
- Historial de pruebas realizadas:
 - Ingresará al historial.
 - El sistema mostrará los registros de las pruebas realizadas con su resultado.
 - El usuario consultará historial y decidirá si descargará el reporte.

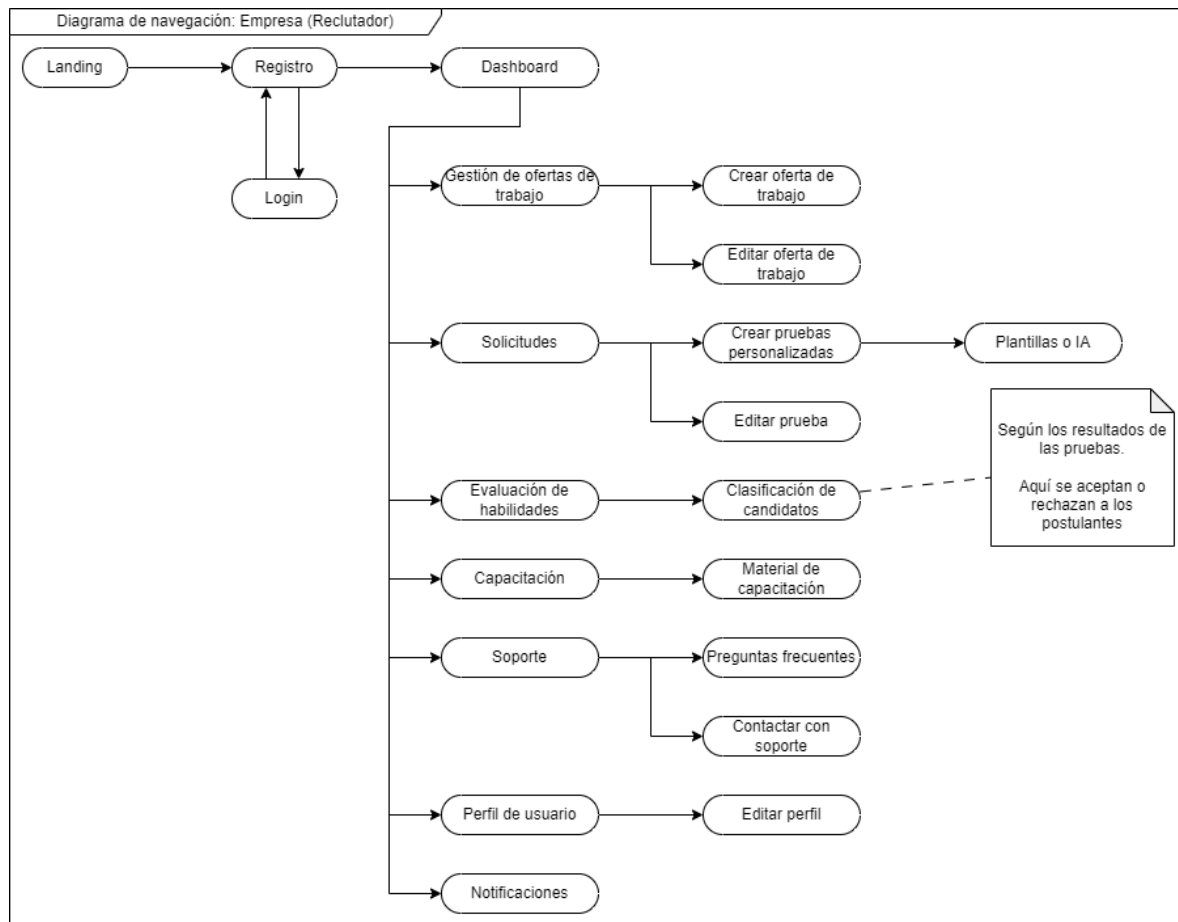


[Ver imagen](#)

Empresa (Experiencia de usuario)

- La empresa se registra previamente en la plataforma.
- Inicia sesión con su cuenta.
- Al ingresar al sistema podrá.
 - Proceso de selección de postulantes:
 - Recibirá solicitudes de postulantes.
 - Creará pruebas de capacidades.
 - Podrá modificar objetivos a la prueba.
 - Tendrá la opción de reclutador conforme (si o no).
 - Seleccionará candidatos finales.
 - Reportes:

- Ingresará a reportes.
- Seleccionará el tipo de reportes.
- Seleccionará generar.
- Recibirá la notificación de reporte listo.
- Seleccionará descargar.
- Tendrá la opción de gestión de usuarios:
 - Ingresará a gestionar datos.
 - Ingresará datos nuevos.
 - Como empresa, almacenará datos ingresados.
 - El sistema mostrará notificación de estado.
- Gestión de ofertas de trabajo:
 - Ingresará a gestionar ofertas de empleo.
 - Podrá añadir, editar o eliminar la oferta de trabajo seleccionada.
 - Ingresará datos de la oferta de trabajo.
 - El sistema validará los datos (si o no).
 - Recibirá la notificación.



[Ver imagen](#)

3.5. Fidelización

La fidelización de los usuarios en la plataforma inteligente de reclutamiento se sustentará en la entrega de una solución que responde de manera precisa y eficiente a las necesidades tanto de las empresas como de los candidatos freelance.

Al ofrecer una evaluación objetiva y automatizada de las habilidades, la plataforma garantizará que las decisiones de contratación se basen en datos confiables, lo que generará confianza en los resultados y reduce la posibilidad de errores; además, la plataforma simplificará significativamente los procesos de reclutamiento, ahorrando tiempo y recursos a ambas partes. Esta combinación de precisión, eficiencia y facilidad de uso creará una experiencia positiva para todos los usuarios, fomentando su lealtad a la plataforma a largo plazo.

3.6. Promoción

La clave para la promoción orgánica de la plataforma radica en la calidad de la experiencia del usuario. Cuando las empresas interactúen con la plataforma, experimentará una serie de beneficios que los motivaron a recomendarla; es decir, la precisión de las evaluaciones, la rapidez de los procesos y la facilidad de uso generan satisfacción y confianza.

Al encontrar de manera eficiente el talento adecuado y al obtener resultados precisos en las evaluaciones, los usuarios se convertirán en verdaderos embajadores de la plataforma, compartiendo sus experiencias positivas con colegas y contactos. Esta promoción boca a boca es invaluable, ya que reforzará la reputación de la plataforma y atraerá a nuevos usuarios que buscan una solución de reclutamiento confiable y eficiente.

3.7. Diagramación

Diagrama de usuarios:

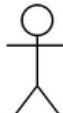
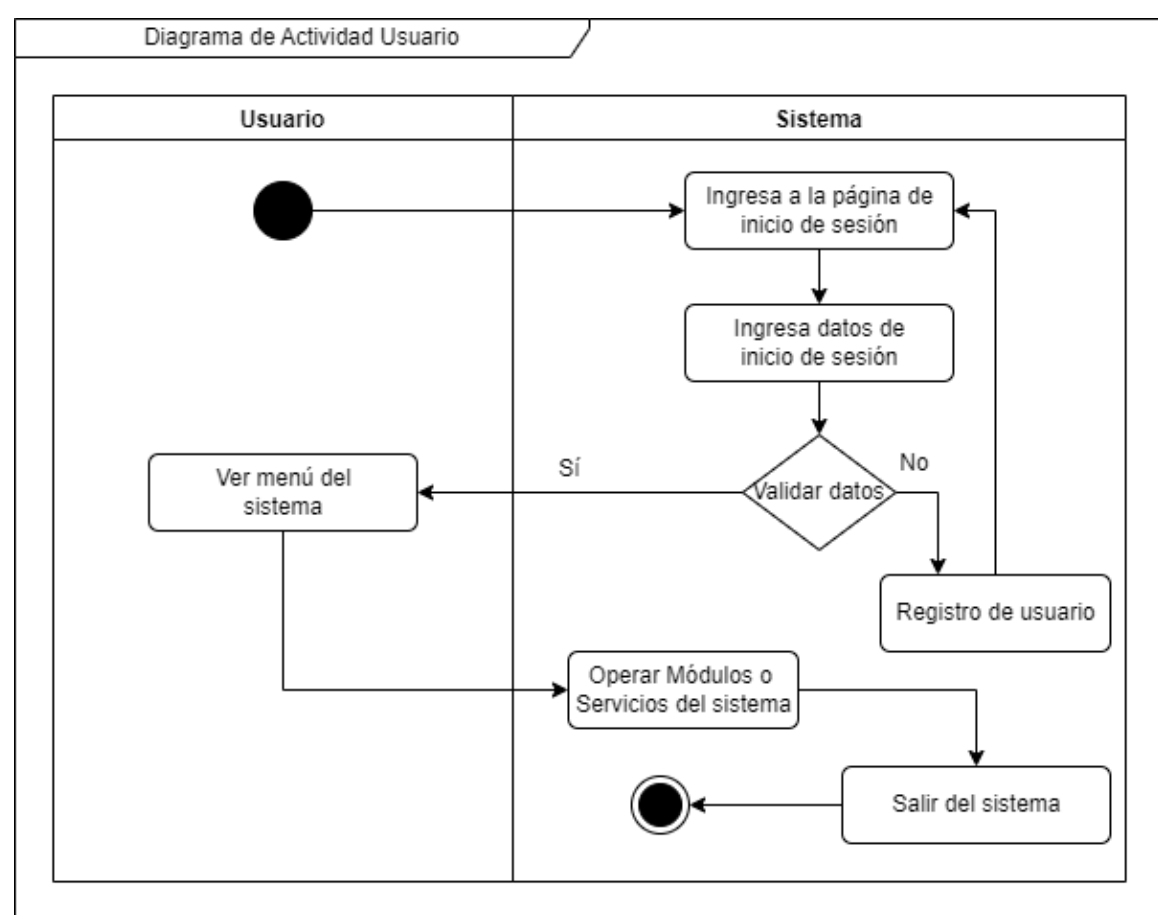
Usuarios	Funciones	Observaciones
 Administrador (Panel de control)	• Gestionar funciones del sistema. • Manejar datos de usuarios. • Soporte y capacitación.	• El administrador puede añadir, editar y eliminar material para la capacitación.
 Freelancer (Candidato)	• Gestionar datos básicos y perfil. • Gestión de freelancer. • Gestión de ofertas. • Evaluación de Habilidades (Pruebas y resultados). • Soporte. • Capacitación.	• El usuario ingresa datos laborales, portafolios, resúmenes curriculares y servicios ofrecidos en la plataforma y de acuerdo a esos datos el sistema lo clasificará para las ofertas de empleo que mejor se adecue. • El usuario realiza evaluaciones de habilidades creadas por las empresas y de acuerdo a sus resultados será elegible o no para el empleo.
 Empresa (Reclutador)	• Gestión de ofertas • Evaluación de Habilidades • Sistema de Recomendaciones Inteligente. • Soporte. • Capacitación.	

Diagrama de actividad del usuario: General.

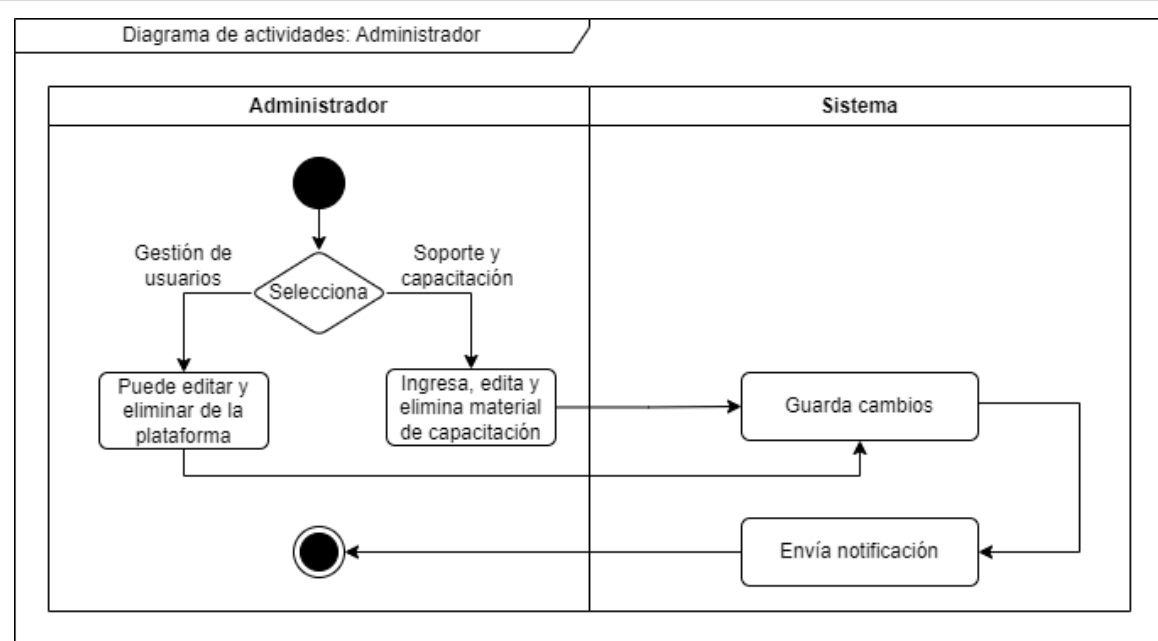


Descripción del diagrama:

1. El usuario ingresa a la pantalla de inicio de sesión
2. El usuario ingresa los datos de inicio de sesión
3. Se validan los datos ingresados
 - a. Si son válidos, se muestra el menú del sistema
 - i. El usuario opera los módulos del sistema
 - ii. El usuario sale del sistema

b. Si no lo son, se retorna a la página de inicio de sesión

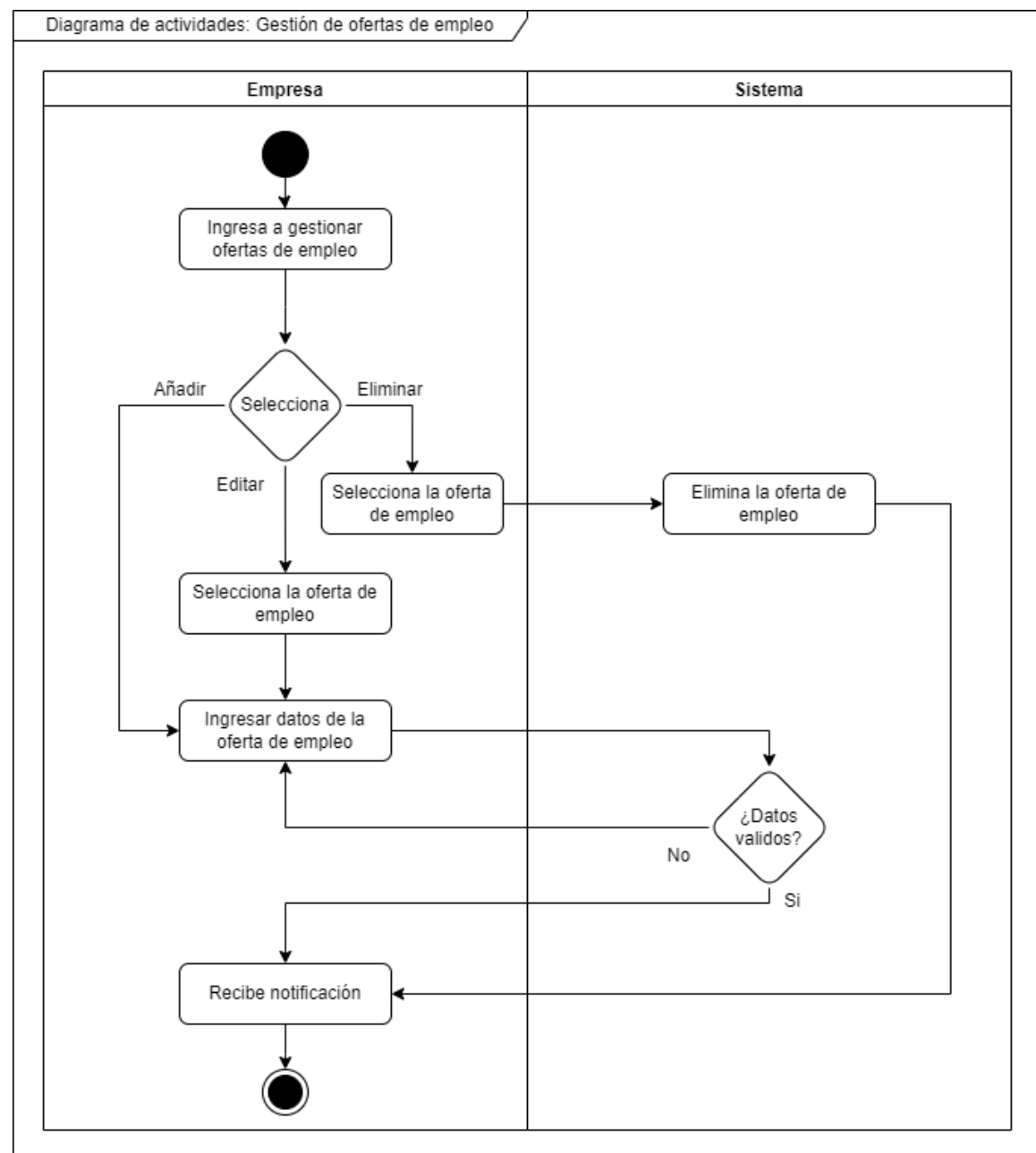
Diagrama de actividad del usuario: Administrador.



Descripción del diagrama:

- El administrador ingresa a la plataforma.
 - Si ingresa a gestión de usuarios:
 - Puede editar y eliminar usuarios de la plataforma.
 - Si ingresa soporte y capacitación:
 - Puede ingresar, editar y eliminar material de capacitación en la plataforma.
 - El sistema guarda los cambios y envía notificaciones de éxito.

Diagrama de actividad del usuario: Gestión de ofertas de empleo.

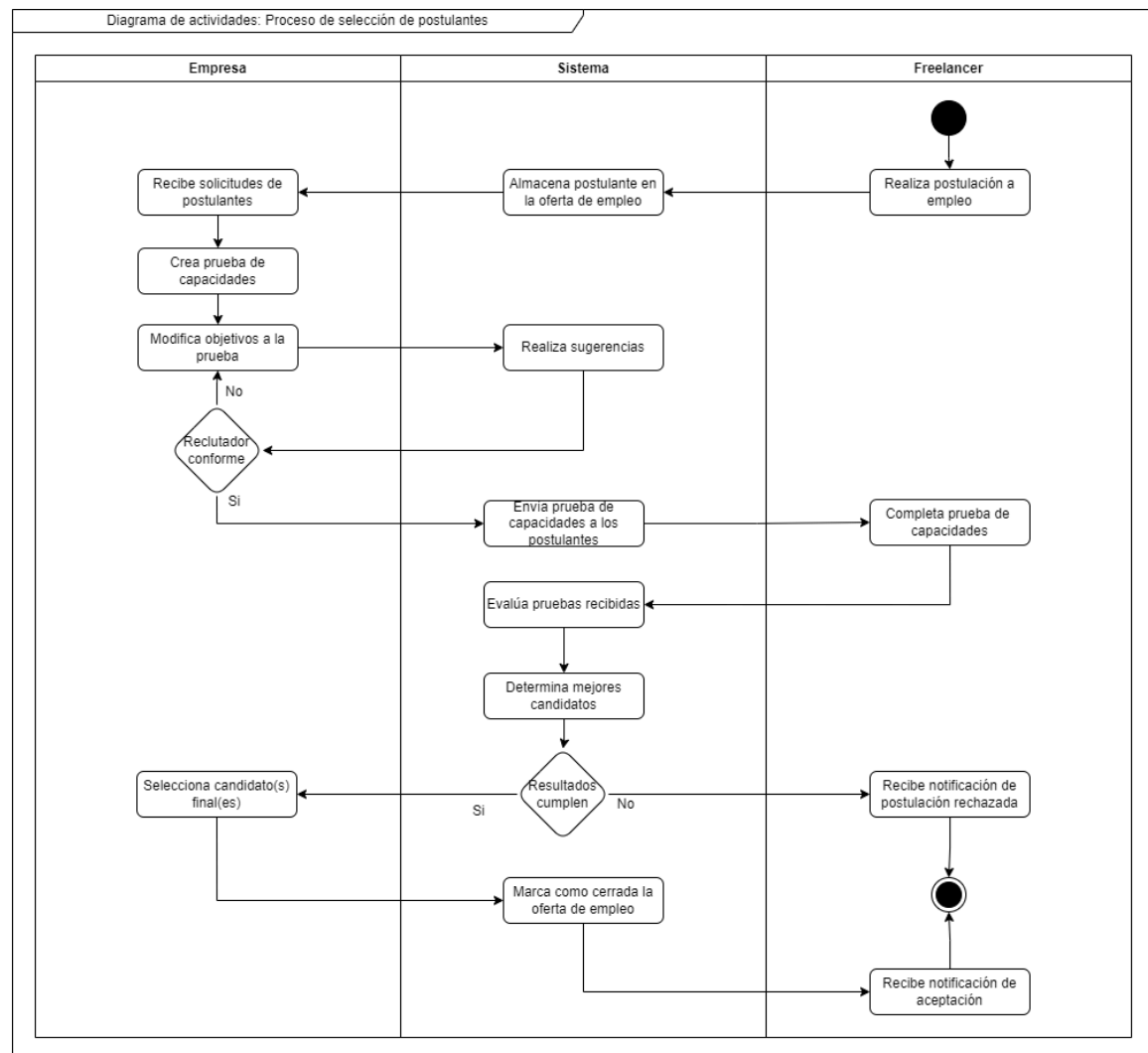


Descripción del diagrama:

- El usuario ingresa a gestionar ofertas de trabajo.
 - Si selecciona Añadir:
 - Ingresar datos de la oferta de empleo.
 - El sistema valida los datos.
 - Si no son válidos:
 - Vuelve a ingresar datos de la oferta de empleo.

- Si son válidos:
 - El usuario recibe notificación de éxito.
- Si selecciona editar:
 - Selecciona la oferta de empleo que desea editar.
 - Ingresa datos de la oferta de empleo.
 - El sistema valida los datos.
 - Si no son válidos:
 - Vuelve a ingresar datos de la oferta de empleo.
 - Si son válidos:
 - El usuario recibe notificación de éxito.
- Si selecciona eliminar:
 - El usuario selecciona la oferta de empleo que desea eliminar.
 - El sistema elimina la oferta de empleo después de que el usuario confirma que está seguro.
 - El usuario recibe notificación.

Diagrama de actividad del usuario: Proceso de selección de postulantes.

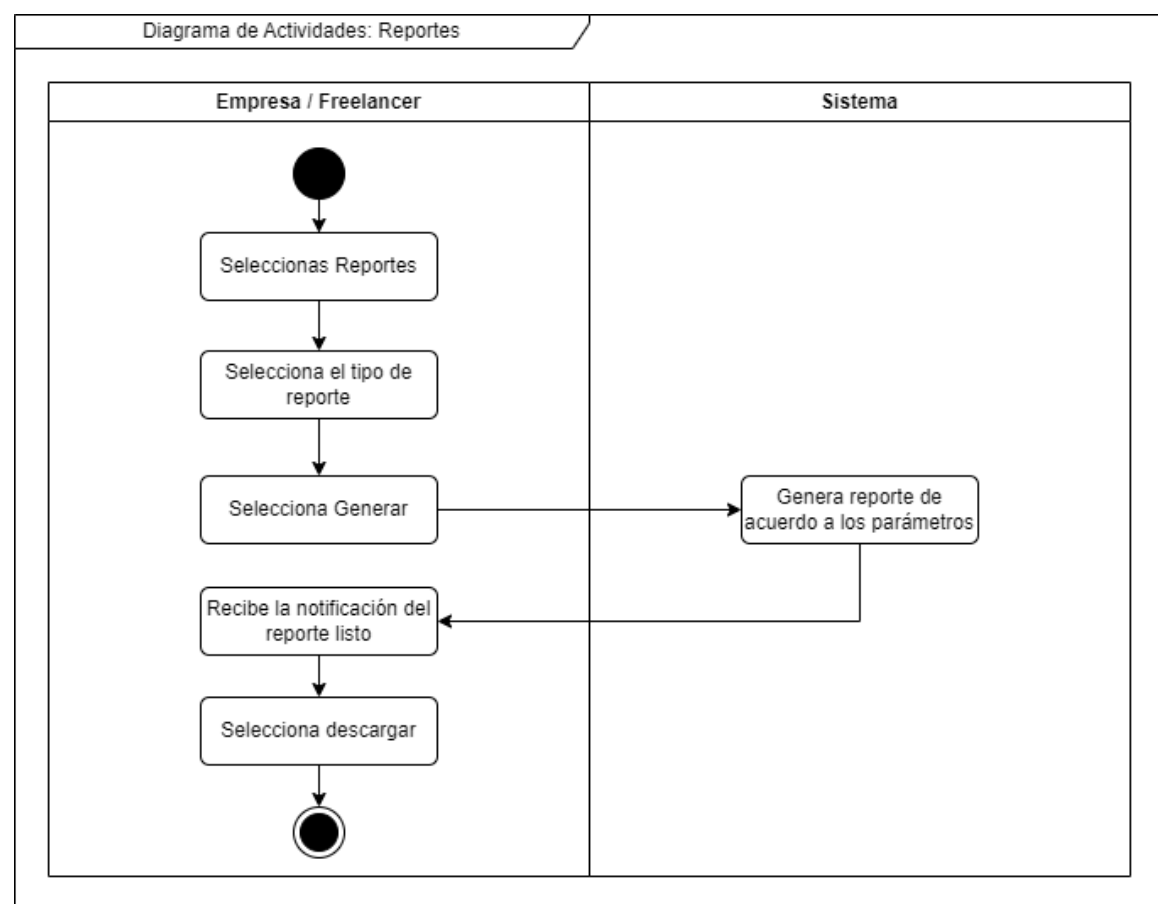


Descripción del diagrama:

- El freelancer realiza postulación de empleo.
- El sistema almacena postulantes en la oferta de empleo.
- La empresa recibe solicitudes de postulantes.
- La empresa crea pruebas de capacidades.
- La empresa modifica objetivos de la prueba.
- El sistema realiza sugerencias por medio de IA.
- Si la empresa no está conforme:
 - La empresa modifica los objetivos de la prueba nuevamente.
- Si la empresa está conforme:
 - El sistema envía prueba de capacidades a los postulantes.
- El freelancer completa la prueba de capacidades.
- El sistema evalúa las pruebas recibidas y determina los mejores candidatos.

- Si los resultados no cumplen:
 - El Freelancer recibe notificación de postulación rechazada.
- Si los resultados cumplen:
 - La empresa selecciona candidato(s) final(es).
 - El sistema marca como cerrada la oferta de empleo y el freelancer recibe notificación de aceptación.

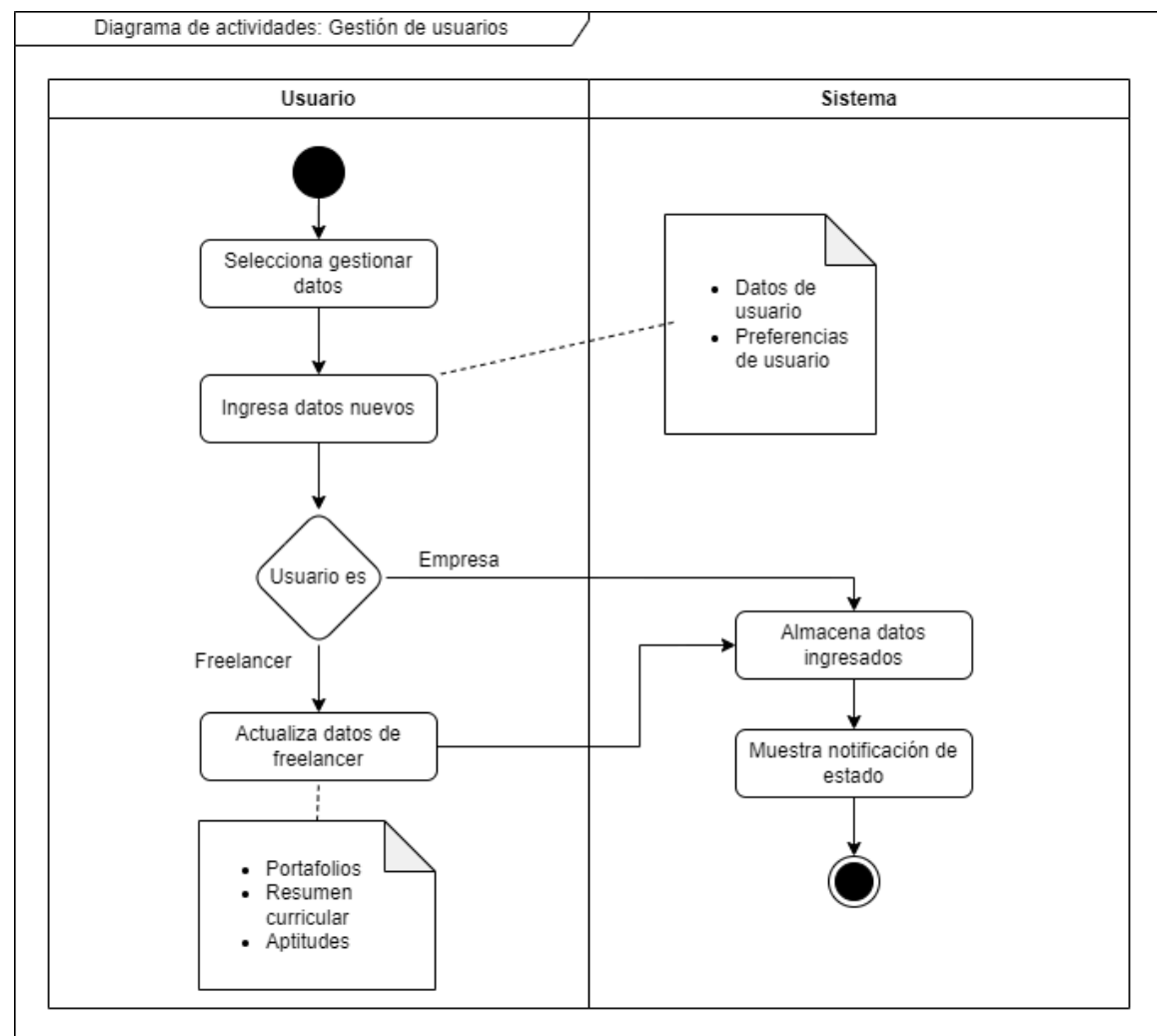
Diagrama de actividad del usuario: Reportes.



Descripción del diagrama:

- El usuario, puede ser la empresa o el freelancer, selecciona reportes.
- El usuario selecciona el tipo de reporte y selecciona generar.
- El sistema genera reporte de acuerdo a los parámetros dados.
- El usuario recibe notificación de reporte listo y selecciona descargar de desearlo.

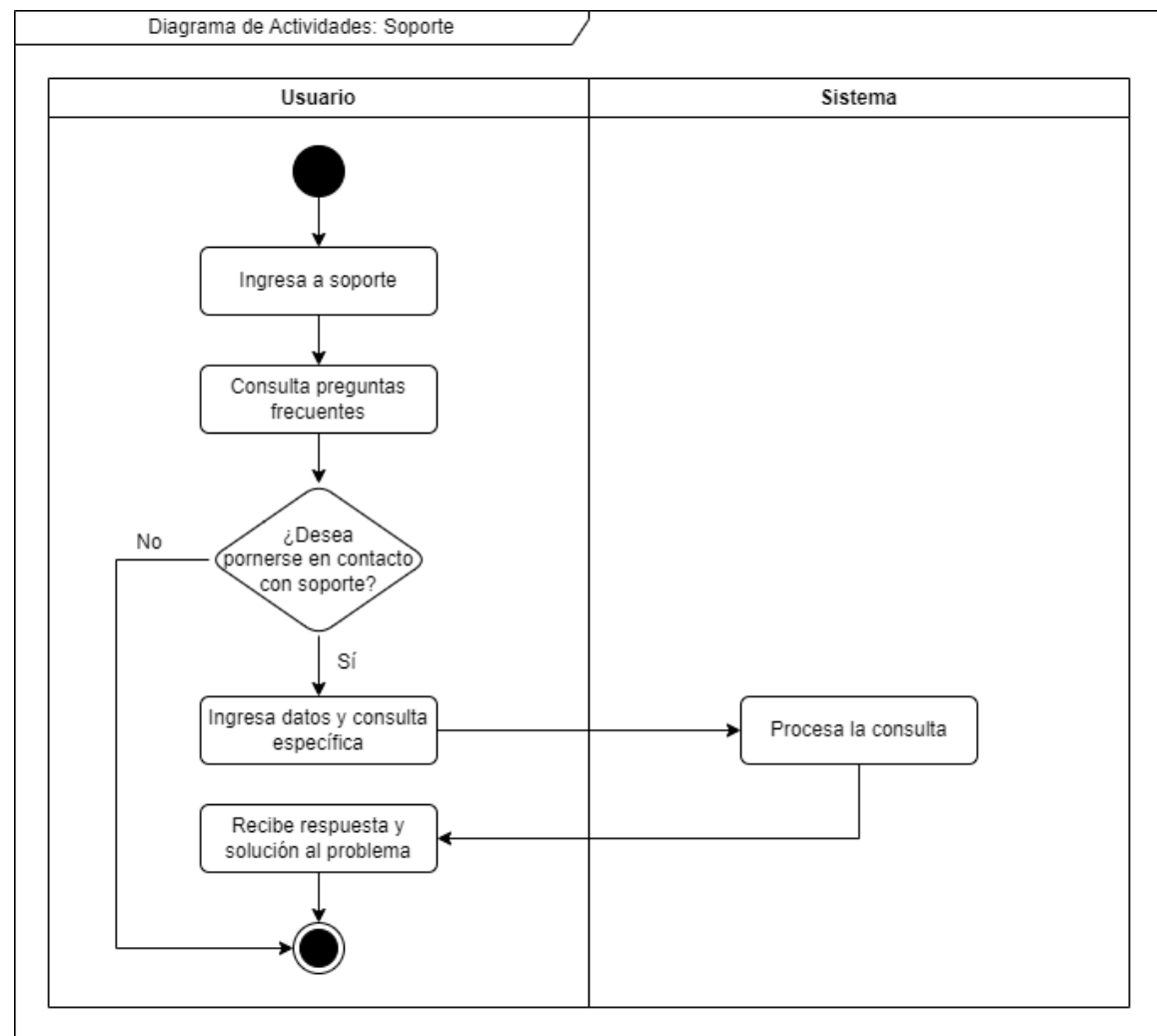
Diagrama de actividad del usuario: Gestión de usuarios.



Descripción del diagrama:

- El usuario selecciona gestionar datos.
- El usuario ingresa datos nuevos, los cuales son sus datos de usuario básicos y sus preferencias.
 - Si el usuario es empresa:
 - El sistema almacena los datos ingresados y muestra notificación de estado.
 - Si el usuario es un freelancer:
 - Actualiza sus datos de freelancer, los cuales son su portafolio, su resumen curricular y sus actitudes laborales.
 - El sistema guarda los datos ingresados y muestra notificación de estado.

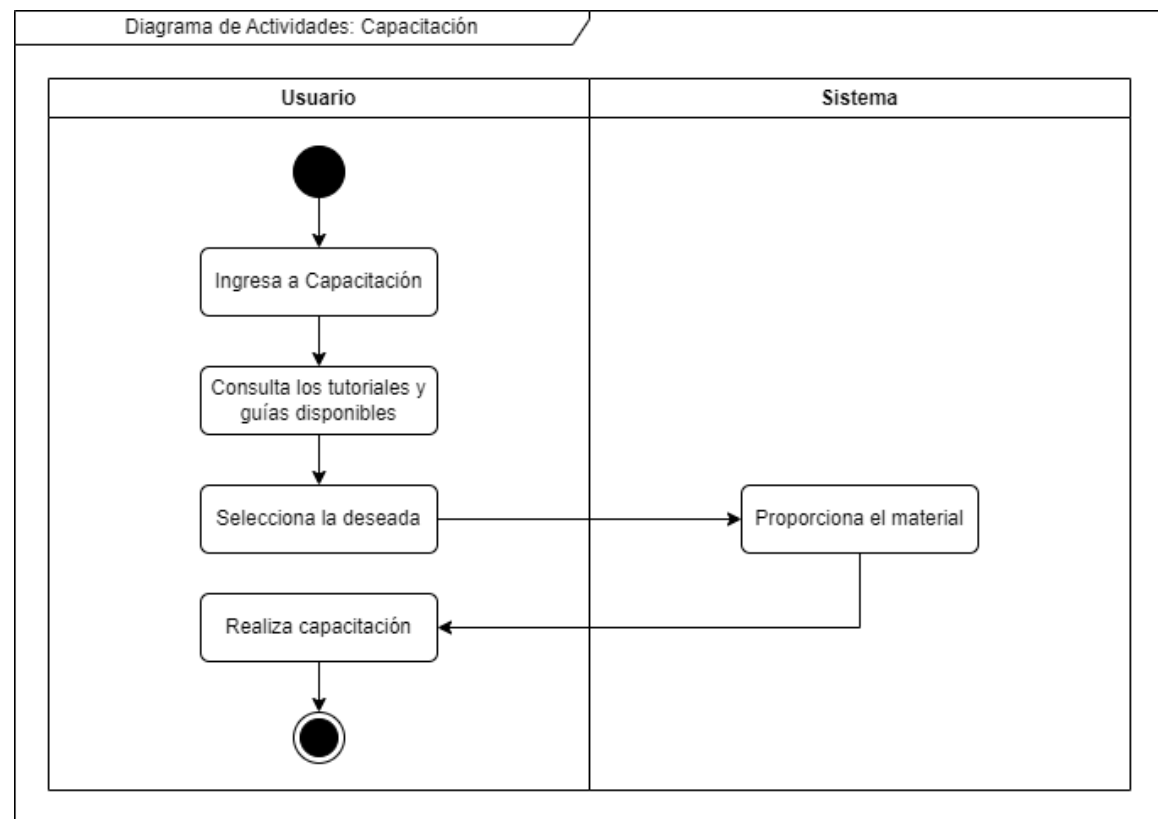
Diagrama de actividad del usuario: Soporte.



Descripción del diagrama:

- El usuario ingresa soporte.
- Consulta las preguntas frecuentes.
- Si desea ponerse en contacto con soporte:
 - Ingresa los datos y consulta específica que necesita.
 - El sistema procesa la consulta.
 - El usuario recibe respuesta y solución al problema.

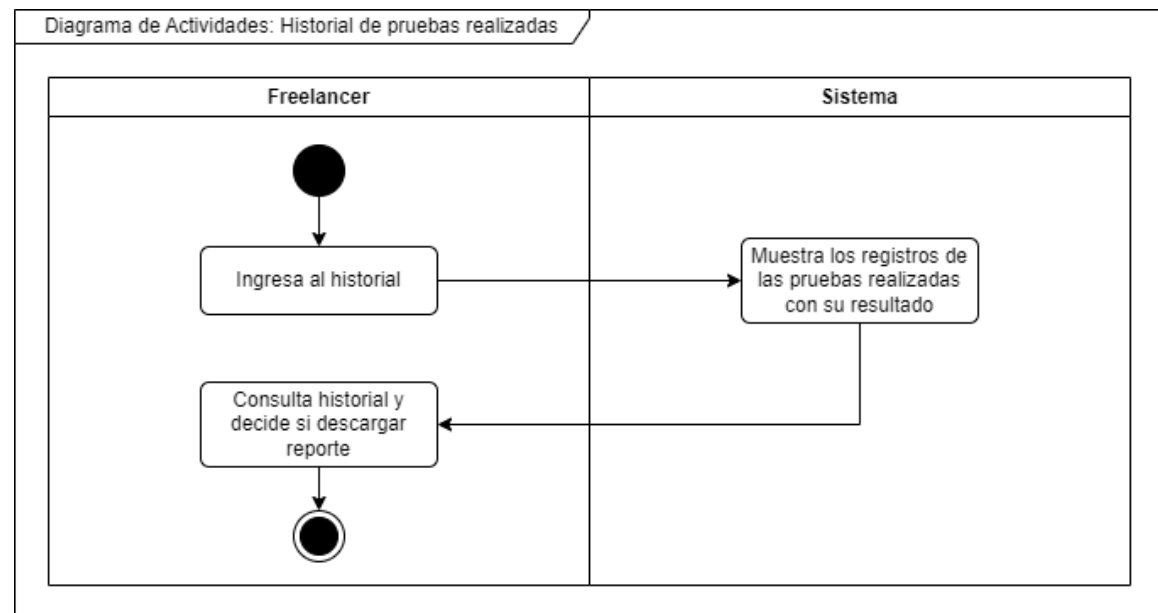
Diagrama de actividad del usuario: Capacitación.



Descripción del diagrama:

- El usuario ingresa capacitación.
- El usuario consulta los tutoriales y guías disponibles.
- Selecciona la que desee según su necesidad.
- El sistema proporciona el material solicitado.
- El usuario realiza la capacitación.

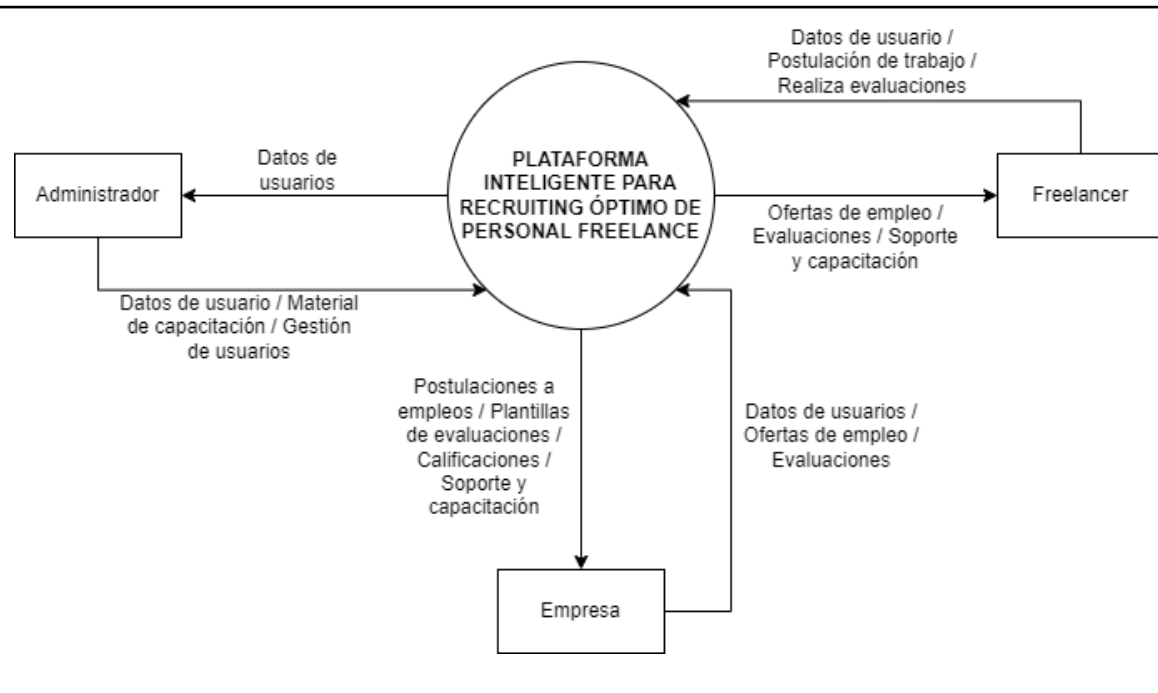
Diagrama de actividad del usuario: Historial de pruebas.



Descripción del diagrama:

- El freelancer ingresa al historial de pruebas.
- El sistema muestra los registros de las pruebas de habilidad realizadas por el usuario junto con su resultado.
- El freelancer consulta el historial y decide si descargar el reporte de las pruebas.

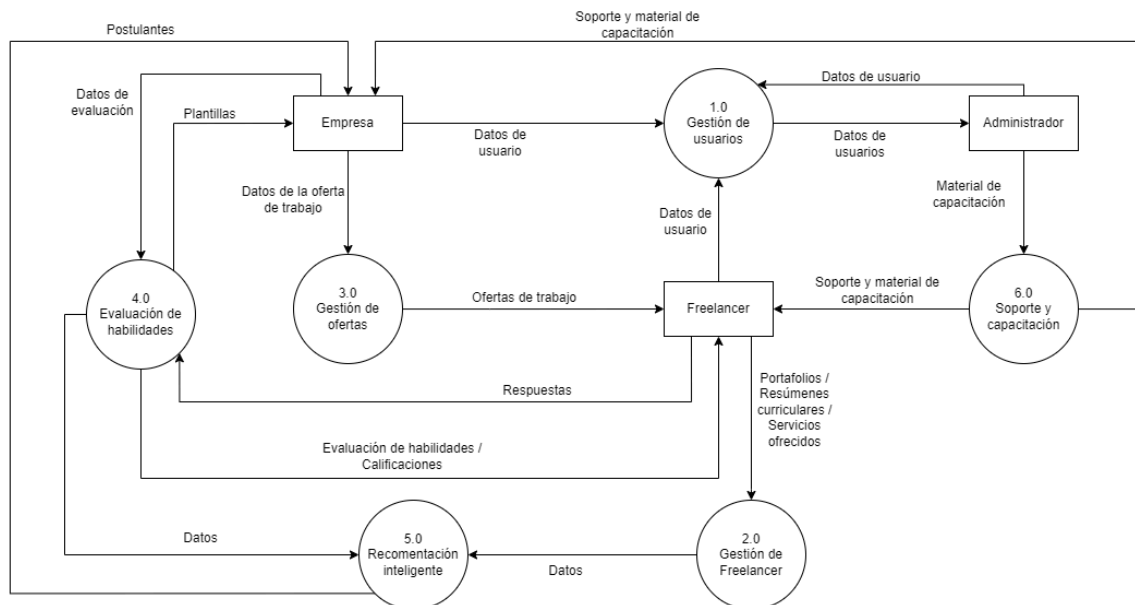
Diagrama de flujo de datos. Nivel 0



Descripción del diagrama:

- El administrador ingresa sus datos de usuario, material de capacitación y gestión de usuarios en la plataforma y la plataforma le proporciona los datos de los usuarios a gestionar.
- El freelancer ingresa sus datos de usuario, postulaciones a trabajos y realiza evaluaciones en la plataforma y la plataforma le proporciona ofertas de empleo, evaluaciones o capacitación.
- La empresa ingresa datos de usuario, ofertas de empleo y evaluaciones en la plataforma y la plataforma le proporciona postulaciones a empleos por parte de los freelancers, plantillas de evaluaciones, calificaciones, soporte y capacitación.

Diagrama de flujo de datos: Nivel 1



Descripción del diagrama:

- El administrador ingresa datos de usuarios a gestión de usuarios, el cual le retorna datos del resto de los usuarios.
- El administrador ingresa material de capacitación a soporte y capacitación.
- El freelancer ingresa portafolios, resúmenes curriculares y servicios ofrecidos a gestión de freelancer, el cual proporciona dichos datos a recomendación inteligente, el cual le recomienda postulantes a las empresas de acuerdo a sus requerimientos.
- La empresa ingresa datos de oferta de trabajo a gestión de ofertas, los cuales le proporcionan estas ofertas a los freelancers.
- La plataforma le proporciona plantillas a través de Las evaluaciones de habilidades a la empresa, la cual con dichas plantillas crea evaluaciones ingresando los datos de las mismas en evaluación de habilidades.
- La evaluación de habilidades proporciona evaluaciones y calificaciones a los freelancers, los cuales ingresan sus respuestas en evaluación de habilidades.
- Evaluación de habilidades, proporciona datos de los resultados a recomendación inteligente para aportar una mejor recomendación a las empresas.
- Soporte y capacitación le proporciona soporte y material de capacitación tanto a los freelancers como a las empresas.

Acceso a la carpeta con la diagramación

[Acceder](#)

3.8 Wireframe

FLUJO DE PANTALLAS

Página de llegada

- Cabecera con logotipo y menú de navegación
- Sección principal para mostrar recursos y funcionalidades
- Sección de registro y configuración del perfil

Panel de control admin

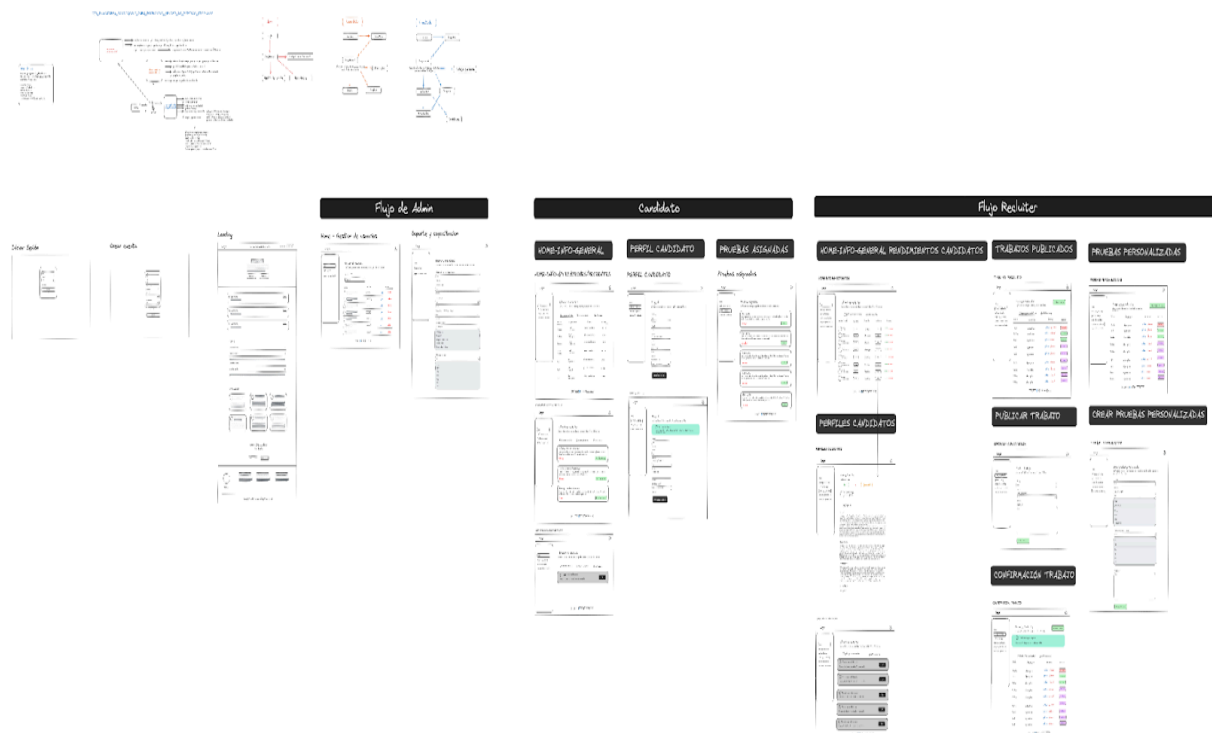
- Cargar capacitaciones
- Gestión de usuarios
- Herramientas

Panel de control de reclutador

- Perfil de los aplicantes
- Pruebas y resultados
- Publicar trabajos
- Notificaciones

Panel de control del candidato

- Perfil de la prueba
- Evaluaciones
- Notificaciones



[Ver imagen completa](#)

3.9 UI

Implementación del wireframe en Figma.

[Ver figma completo](#)

3.10 Arquitectura del Proyecto

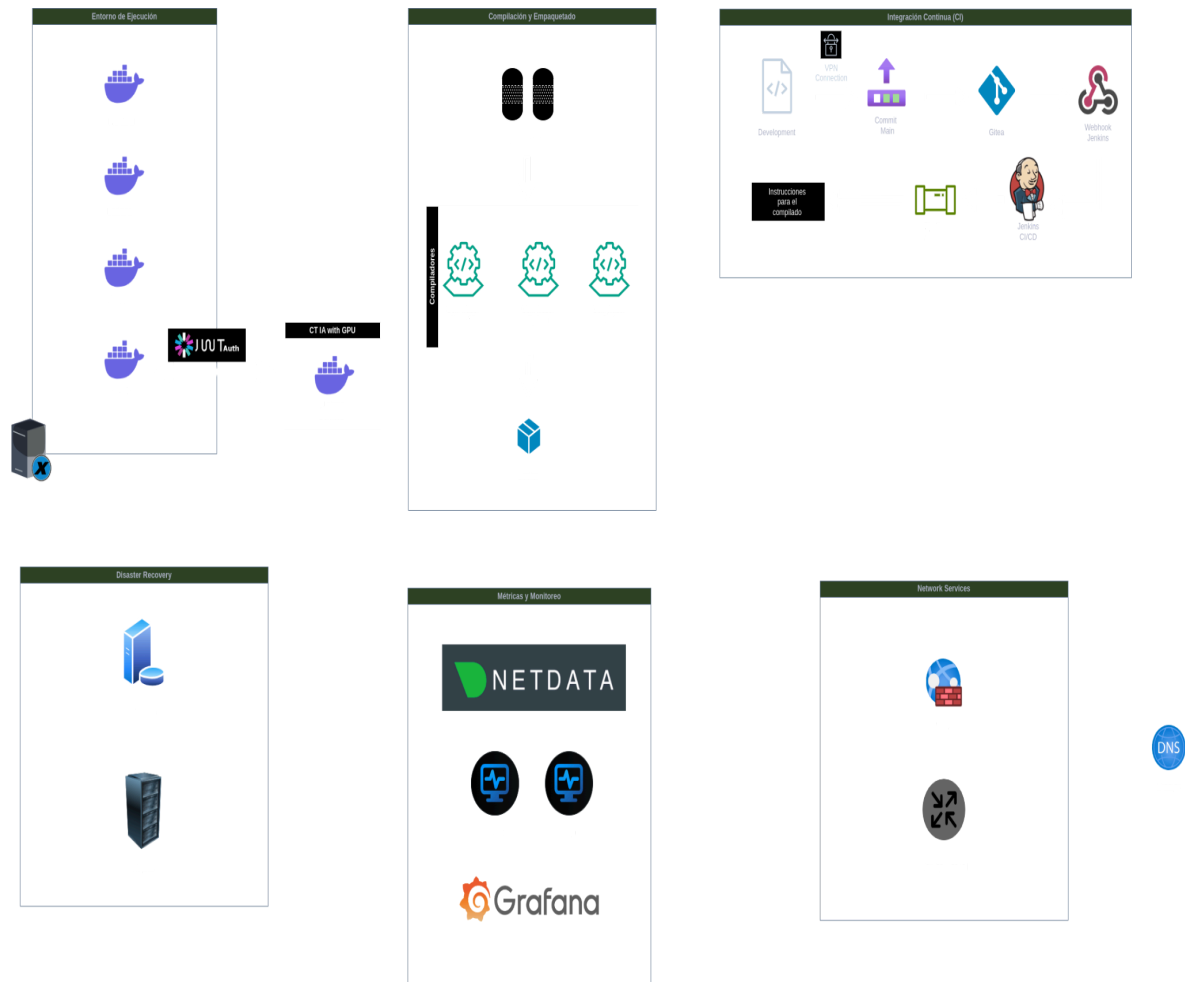
La plataforma inteligente de reclutamiento se basa en un diseño de microservicios, que permite a cada servicio o aplicación ser desarrollada y escalada independientemente.

Componentes

- **API Gateway:** La puerta de entrada principal del sistema, responsable de recibir solicitudes y distribuirlas entre los diferentes servicios.
- **Render Service:** El servicio encargado de realizar la renderización del material audiovisual. Utiliza tecnologías como Rust con Python para implementar algoritmos de aprendizaje automático.
- **IA/ML Service:** El servicio que utiliza inteligencia artificial y machine learning para analizar el material audiovisual y predecir resultados.
- **Database Service:** El servicio responsable de almacenar y gestionar los datos del proyecto, incluyendo la información de proyectos, usuarios y versiones.
- **Storage Service:** El servicio que almacena y gestiona los archivos multimedia producidos por el sistema.
- **Frontend Service:** El servicio que se encarga de presentar la interfaz de usuario para los usuarios finales, utilizando tecnologías como React.
- **Worker Service:** El servicio que se ocupa de realizar tareas en segundo plano, como la renderización y procesamiento de archivos.

Despliegue

El proyecto se despliega en una infraestructura de nube pública, utilizando los servicios de Sinergy. Cada componente del sistema se encuentra en un contenedor Docker, lo que facilita el despliegue y la escalabilidad.



[Ver imagen completa](#)

IV. Metodología de desarrollo

4.1 Introducción

La metodología Lean Development:

Se ha consolidado como un enfoque fundamental en la gestión de proyectos de desarrollo, especialmente en entornos donde la eficiencia y la calidad son cruciales. Esta metodología se basa en principios que buscan maximizar el valor para el cliente mientras se minimizan los desperdicios, permitiendo a los equipos de trabajo adaptarse rápidamente a las necesidades cambiantes del mercado. Al integrar prácticas como la priorización del trabajo, la mejora continua y la visualización del flujo de trabajo, lean development no solo optimiza los procesos, sino que también fomenta una cultura organizacional que valora la retroalimentación constante y el aprendizaje. En un mundo donde la innovación y la agilidad son esenciales, lean development proporciona un marco robusto que permite a las organizaciones entregar productos de alta calidad de manera eficiente, garantizando así una mayor satisfacción del cliente y una ventaja competitiva sostenible.

4.2 Implementación de la Metodología

1. Evaluación inicial del proyecto

La implementación de lean development comienza con una evaluación exhaustiva del estado actual del proyecto. Esto implica:

- **Análisis de procesos:** identificar cuellos de botella y áreas donde se generan desperdicios.
- **Definición de objetivos:** establecer metas claras que alineen las expectativas del cliente con los resultados deseados.

2. Formación del equipo

Es esencial formar un equipo multidisciplinario que incluya miembros con diversas habilidades y experiencias. Este equipo debe recibir capacitación en los principios lean para asegurar una comprensión profunda de la metodología.

3. Priorización del trabajo

Una vez constituido el equipo, el siguiente paso es priorizar el trabajo en función del valor que aporta al cliente. Esto implica:

- **Identificación de funcionalidades críticas:** determinar cuáles son las características o funcionalidades más importantes para el cliente.
- **Planificación ágil:** establecer un plan que permita al equipo concentrarse en completar las tareas más valiosas primero.

4. Visualización del flujo de trabajo

La visualización es clave en lean development. Se recomienda utilizar tableros kanban o herramientas similares para:

- Representar el estado actual: mostrar claramente el progreso de cada tarea.
- Identificar cuellos de botella: facilitar la identificación rápida de problemas en el flujo de trabajo.

5. Eliminación de desperdicio

El equipo debe realizar un análisis continuo para identificar y eliminar desperdicios en el proceso, lo que incluye:

- Revisión regular de procesos: evaluar constantemente las actividades para determinar cuáles no aportan valor.
- Optimización del flujo: ajustar los procesos para mejorar la eficiencia y reducir tiempos muertos.

6. Mejora continua

Fomentar una cultura de mejora continua es fundamental para el éxito a largo plazo. Esto incluye:

- Ciclos regulares de revisión: realizar reuniones periódicas para evaluar el progreso y discutir posibles mejoras.
- Fomento del feedback: establecer canales abiertos para que todos los miembros del equipo puedan compartir sus ideas y sugerencias.

7. Testing continuo

Integrar pruebas desde las primeras etapas del desarrollo es esencial para asegurar la calidad del producto final. Esto implica:

- **Pruebas automatizadas:** implementar pruebas automatizadas que se ejecuten regularmente durante el ciclo de desarrollo.
- **Revisión continua:** realizar revisiones constantes del código para detectar y corregir errores tempranamente.

8. Flexibilidad y adaptación

Finalmente, es crucial mantener una mentalidad flexible y abierta al cambio. La metodología lean development promueve:

- **Ajustes dinámicos:** la capacidad de adaptarse rápidamente a las nuevas necesidades del cliente o cambios en el entorno.
- **Reevaluación constante:** revisar regularmente los objetivos y prioridades para asegurarse de que siguen alineados con las expectativas del cliente.

Desafíos:

- Dificultades en la priorización
 - Complicaciones para identificar qué tareas aportan más valor al cliente.
 - Conflictos entre equipos con diferentes prioridades y objetivos.
- Integración de herramientas y procesos
 - Desafíos en la selección de herramientas adecuadas para visualizar flujos de trabajo.
 - Necesidad de ajustar procesos existentes para integrar lean development.
- Mantenimiento del compromiso
 - Dificultades para mantener el compromiso a largo plazo con los principios lean.
 - Riesgo de agotamiento del equipo debido a la presión por mejorar continuamente.
- Medición del progreso y éxito
 - Complicaciones para definir indicadores clave de éxito y progreso.
 - Desafíos en la recopilación e interpretación de datos relevantes.
- Adaptación a cambios externos
 - Necesidad de flexibilidad ante condiciones del mercado y necesidades cambiantes del cliente.
 - Dificultades para mantener agilidad en un entorno competitivo.

V. Desarrollo de la tecnología

5.1 Introducción

El objetivo principal de este proyecto es desarrollar una plataforma inteligente para el reclutamiento óptimo de personal freelance. Con el fin de aumentar la eficacia y reducir los costos en el proceso de postproducción.

En este capítulo, se presentará el desarrollo tecnológico detrás del proyecto, incluyendo la implementación de APIs, la creación de una base de datos robusta y la integración con terceros servicios. El objetivo es brindar una visión general y detallada del proceso de desarrollo tecnológico que nos llevó a crear la plataforma.

En la sección siguiente describiremos en detalle el desarrollo tecnológico del proyecto, incluyendo el diseño e implementación de las componentes clave.

5.2 Diseño e implementación

Frontend

Utilizando React, este proyecto se beneficia de una biblioteca avanzada para aplicaciones web, ideal para crear una experiencia de usuario fluida y de alta interactividad en entornos virtuales.

React se destaca por su capacidad de construir interfaces de usuario eficientes y responsivas, lo que es crucial para cualquier plataforma que busca ofrecer una experiencia de usuario óptima. Su enfoque en la declaratividad y el renderizado eficiente asegura que los usuarios experimenten transiciones suaves y una interacción continua, sin demoras, al navegar por diferentes escenarios.

TypeScript para un desarrollo sólido: La implementación de TypeScript mejora la calidad del código y facilita la detección de errores, lo que es fundamental en el desarrollo de aplicaciones complejas. Esta combinación permite un desarrollo más robusto y mantenible, asegurando que la complejidad inherente a aplicaciones avanzadas se maneje de manera eficiente.

Tailwind CSS para una interfaz intuitiva y adaptativa: El uso de Tailwind CSS asegura una interfaz de usuario coherente y visualmente atractiva, lo que es vital para mantener a los usuarios comprometidos. Su diseño responsivo se adapta perfectamente a diferentes dispositivos, mejorando la accesibilidad y la experiencia de usuario en cualquier aplicación.

Vite.js para un desarrollo y construcción eficientes: Vite.js, seleccionado por su velocidad y eficiencia, se integra con TypeScript para proporcionar un entorno de desarrollo ágil. Esto permite un ciclo de desarrollo más rápido, con tiempos de carga instantáneos y feedback rápido para los desarrolladores, lo que es crucial para iterar rápidamente en el diseño y funcionalidad de cualquier plataforma.

Zustand para una gestión de estado eficiente: Zustand, seleccionado por su simplicidad y flexibilidad, se integra con React y TypeScript para proporcionar una gestión de estado ágil. Esto permite un manejo del estado global más intuitivo, con una API mínima y sin complicaciones para los desarrolladores, lo que es significativo para mantener la claridad y eficiencia en la lógica de estado de cualquier aplicación.

Yarn para la gestión eficiente de paquetes: La adopción de Yarn mejora el proceso de desarrollo al optimizar el almacenamiento y el rendimiento de la instalación de dependencias. Esto facilita el mantenimiento y la escalabilidad del proyecto, asegurando una base de código limpia y una integración sin problemas de nuevas tecnologías o bibliotecas.

[Visitando este enlace podrás tener acceso a todas las imágenes del proyecto](#)

Fragmento de código en donde se aprecia la estructura de manejo de errores en el frontend.

```
1  const createErrorFactory = (name: string, message?: string) => {
2    return class CustomizedError extends Error {
3      constructor(error?: Error) {
4        super(error?.message || message)
5        this.name = name
6        this.stack = ''
7      }
8    }
9  }
10
11  export const ConnectionError = createErrorFactory(
12    'ConnectionError',
13    'Conexión fallida. No se pudo establecer conexión con el servidor' "Conexión": Unknown word.
14  )
15  export const ValidationError = createErrorFactory('ValidationError')
16  export const AuthenticationError = createErrorFactory('AuthenticationError', 'Email o contraseña inválidos') "contraseña": Unknown word.
17  export const AuthorizationError = createErrorFactory('AuthorizationError', 'No tiene permisos suficientes') "tiene": Unknown word.
18  export const NotFoundError = createErrorFactory('NotFoundError', 'Página no encontrada') "Página": Unknown word.
19  export const TimeoutError = createErrorFactory('TimeoutError', 'Error de timeout. Intente de nuevo más tarde') "Intente": Unknown word.
20  export const ConflictError = createErrorFactory('ConflictError', 'Ya existe un usuario con esas credenciales') "existe": Unknown word.
21  export const ServerError = createErrorFactory('ServerError')
22  export const ClientError = createErrorFactory(
23    'ClientError',
24    'Parse recibió un JSON inválido. Puede haber errores de sintaxis, como comillas mal colocadas, comas faltantes, o falta de llaves de apertura o c
25  )
26  export const DatabaseError = createErrorFactory('DatabaseError')
27
```

[Ver imagen](#)

```
1 import {
2   ⚠️ ConnectionError,
3   ValidationError,
4   AuthenticationError,
5   AuthorizationError,
6   NotFoundError,
7   TimeoutError,
8   ConflictError,
9   ServerError,
10  ClientError,
11  DatabaseError,
12 } from './factory'
13
14 const errorHandler = (error: unknown) => {
15   if (error instanceof Error) {
16     if (error.message === 'XMLHttpRequest failed: "Unable to connect to the Parse API"') {
17       throw new ConnectionError()
18     }
19     if (error.message.includes('XMLHttpRequest failed:')) {
20       throw new ConnectionError(error)
21     }
22     if (error.message.includes('Received an error with invalid JSON from Parse:')) {
23       throw new ServerError(error)
24     }
25     if (error.message === '') {
26       throw new ValidationError()
```

[Ver imagen](#)

Fragmento de código en donde se aprecia el manejo encapsulado de Hooks, en este caso las imágenes muestran los hooks que usamos para el inicio de sesión del usuario.

```
import { Moralis } from "moralis-v1";
import { useState } from "react";
import type { LogIn } from "../types";
import handleError from "@errors";

function useLogIn() {
  const [loading, setLoading] = useState(false);

  const logIn = async ({ email, password }: LogIn) => {
    setLoading(true);

    try {
      const response = await Moralis.User.logIn(email, password);
      if (response.id) {
        return { success: "Registro exitoso" };
      }
    } catch (error) {
      return handleError(error);
    } finally {
      setLoading(false);
    }
  };

  return {
    logIn,
    loading,
  };
}

export default useLogIn;
```

En la siguiente imagen se muestra el componente con el formulario de autenticación de usuario

> node_modules	12	
> public	13	export default function LogInForm() {
✓ src	14	const { logIn, loading } = useLogIn()
> assets	15	const navigate = useNavigate()
> config	16	const { showBoundary } = useErrorBoundary()
> errors	17	const formik = useFormik({
> layout	18	initialValues: {
> logging	19	email: '',
> mockData	20	password: '',
✓ modules	21	} as LogIn,
✓ auth	22	validationSchema: LogInSchema,
✓ components	23	onSubmit: async ({ email, password }) => {
✓ form	24	try {
LogInForm.tsx	25	const result = await logIn({
signUpForm.tsx	26	email,
authError.tsx	27	password,
> enum	28	})
> hooks	29	if (result?.success) navigate(Routes.home)
> lib	30	} catch (error) {
> schemas	31	return showBoundary(error)
> types	32	}
> utils	33	},
> core	34	})
> users	35	
> routes	36	return (
> screens	37	<main className="">
> services	38	<div className="ooo">
> store	39	Iniciar sesión
> styles	40	</div>
> types	41	<form className="ooo" onSubmit={formik.handleSubmit}>
App.tsx	42	{ /* Input Email */ }
main.tsx	43	<label className="ooo">
OUTLINE	44	Correo
TIMELINE	45	<input
	46	onChange={formik.handleChange}
	47	value={formik.values.email}
	48	onBlur={formik.handleBlur}
	49	name="email"
	50	placeholder="email@gmail.com"

[Ver imagen](#)

En el siguiente fragmento de código mostramos el uso de la biblioteca yup para definir esquemas de validación para formularios de registro e inicio de sesión.

```
import * as yup from 'yup'

const passwordRules =
  /^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#.%&@\$\%^&*])(?={8,})/

const usernameSignUp = /^(S+$)/g
const emailRules = /^[^@]+@^[^@]+\.[^@]+$

export const SignUpSchema = yup.object().shape({
  fullName: yup
```

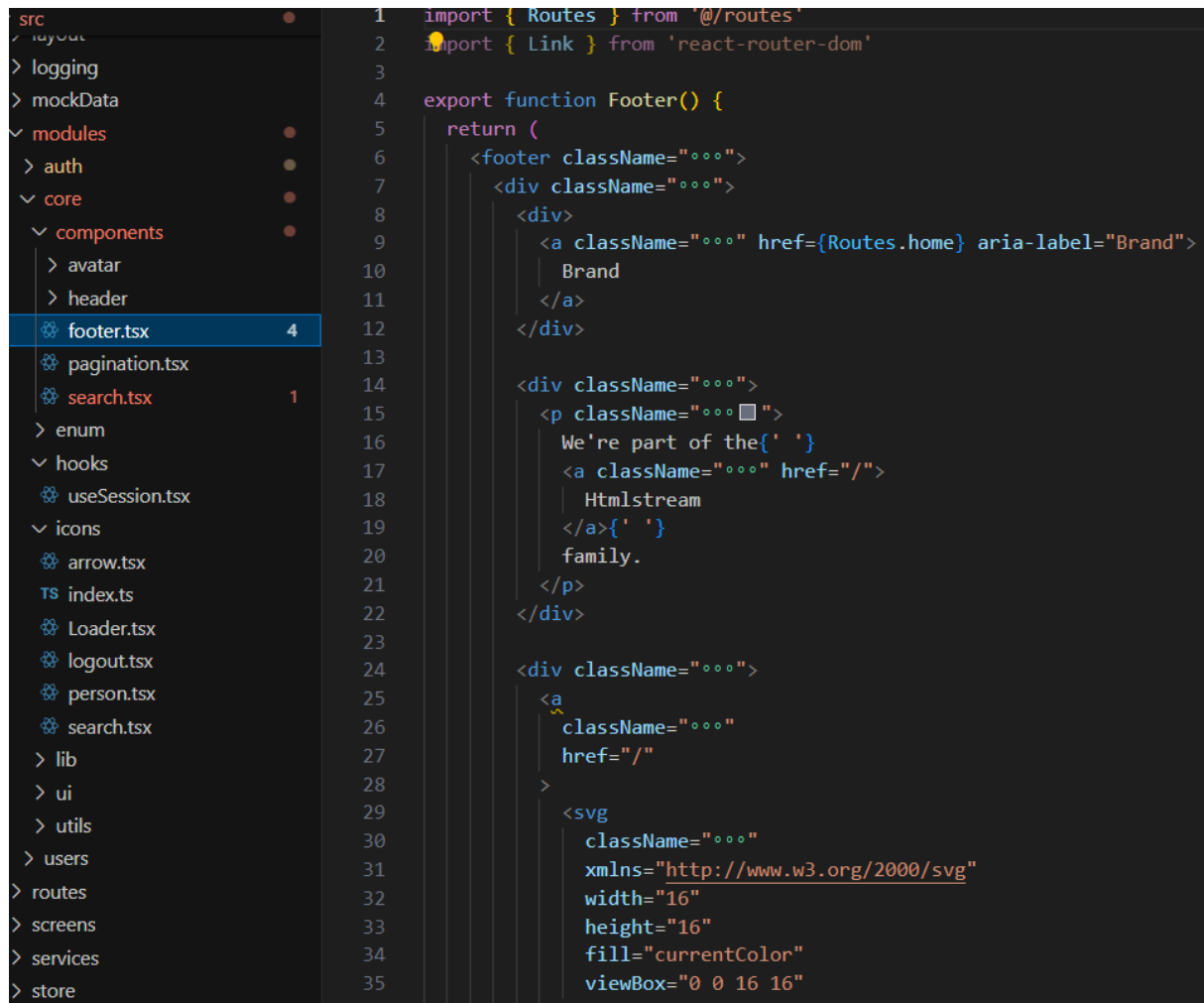
```

        .string()
        .min(5, 'Full Name must be at least 5 characters long')
        .max(65, 'Full name must contain a maximum of 65 characters')
        .required('Required, Please Enter your Full Name'),
    username: yup
        .string()
        .min(5, 'Username must be at least 5 characters long')
        .max(25, 'User name must contain a maximum of 25 characters')
        .required('Required, Please Enter your User Name ')
        .matches(usernameSignUp, 'spaces not allowed'),
    email: yup.string().max(255).email('Must be a valid email').required('Email
is required'),
    password: yup
        .string()
        .required('Please Enter your password')
        .matches(
            passwordRules,
            'Must Contain 8 Characters, One Uppercase, One Lowercase, One Number and
One Special Case Character: ! @ # . * % & @'
        ),
    })

export const LogInSchema = yup.object().shape({
    email: yup
        .string()
        .min(5, 'El nombre de usuario debe contener al menos 5 caracteres')
        .max(25, 'El nombre de usuario debe tener un máximo de 25 caracteres')
        .required('Requerido, por favor introduzca su nombre de usuario')
        .matches(emailRules, 'Debe ser un mail'),
    password: yup
        .string()
        .required('Requerido, por favor ingrese la contraseña')
        .matches(
            passwordRules,
            'Debe contener 8 caracteres, una mayúscula, una minúscula, un numero y
un carácter especial: : ! @ # . * % & @'
        ),
    })

```


En la siguiente imagen podemos apreciar lo que es el core de la aplicación, en este módulo se manejan todos los componentes, hooks y utilidades que se reutilizan en diferentes partes de la aplicación.



[Ver imagen](#)

En el siguiente código se muestra un componente reutilizable que está dentro del core.

```
import { useNavigate } from 'react-router-dom'
import { Search as SearchIcon } from '@modules/core/icons'

export default function Search() {
  const navigate = useNavigate()
  const url = new URL(globalThis.location.href)

  const handlerSubmit = (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault()
  }
}
```

```

const form = new FormData(event.currentTarget)

const q = form.get('q') as string

console.log(q)

q?.length > 0 ? url.searchParams.set('q', q) :
url.searchParams.delete('q')

navigate(url.pathname + url.search)
}

return (
  <form
    onSubmit={(e) => handlerSubmit(e)}
    className="bg-white shadow-lg border rounded-md max-w-lg w-full flex
gap-1 items-center py-1 pr-1 pl-3"
  >
    <input
      placeholder="Buscar"
      type="search"
      className="bg-transparent outline-none w-full"
      defaultValue={String(url.searchParams.get('q') || '')}
      name="q"
    />
    <button type="submit" className="bg-neutral-50 text-neutral-500
rounded-lg p-2">
      <SearchIcon />
    </button>
  </form>
)
}

```

El siguiente código define un hook personalizado en React llamado useSession, que se utiliza para gestionar el estado de la sesión de usuario en la aplicación.

```

import { Routes } from '@routes'
import type { Session, SessionStatus } from '@types'
import Moralis from 'moralis-v1'
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'

```

```

import { $SessionStatus } from '../enum'
// import { getUserById } from '@services/users'

function useSession() {
  const [user, setUser] = useState<Session | null>(null)
  const [status, setStatus] = useState<SessionStatus>($SessionStatus.LOADING)
  const navigate = useNavigate()

  useEffect(() => {
    try {
      const currentSession = Moralis.User.current()

      if (!currentSession?.attributes) {
        setStatus($SessionStatus.UNAUTHENTICATED)
        return
      }

      const { id } = currentSession

      const { sessionToken, email, user_type } = currentSession.attributes

      if (!sessionToken) {
        setStatus($SessionStatus.UNAUTHENTICATED)
        return
      }

      setUser({
        id,
        sessionToken,
        email,
        fullName: '',
        username: '',
        type: user_type,
      })

      // getUserById({ id }).then((response) => {
      //   response.data &&
      //     setUser({
      //       id: currentSession.id,
      //       sessionToken,
      //       email: response.data?.email,

```

```

        //      fullName: response.data?.fullName,
        //      username: response.data?.username,
        //      type: response.data?.type
        //    })
        //  })

        setStatus($SessionStatus.AUTHENTICATED)
      } catch (error) {
        console.error(error)
        setStatus($SessionStatus.UNAUTHENTICATED_ERROR)
      }
      return () => {
        setUser(null)
      }
    }, [])

    const signOut = () => {
      Moralis.User.logout()
      navigate(Routes.logIn)
      setStatus($SessionStatus.UNAUTHENTICATED)
    }

    return {
      user,
      status,
      signOut,
    }
  }
}

export default useSession

```

En el siguiente código se muestra el manejo de rutas de la aplicación.

```

import { useBoundStore } from '@store/index'

import { Navigate, Outlet } from 'react-router-dom'

export default function AdminRoutes({ redirectPath = '/log-in' }) {

  const { User, Authenticated, LoadingRefresh } = useBoundStore()

```

```

    if (LoadingRefresh) {

        return null

    }

    if (!Authenticated) return <Navigate to={redirectPath} replace={true} />

    // Verificar si User existe y si tiene la propiedad userType

    if (User?.user_type && User.user_type !== 'admin') {

        return <Navigate to={redirectPath} replace={true} />

    }

    return <Outlet />

}

```

```

import { useBoundStore } from '@store/index'
import { Navigate, Outlet } from 'react-router-dom'
export default function IsLogged({ redirectPath = '/log-in' }) {
    const { Authenticated, LoadingRefresh } = useBoundStore()
    if (LoadingRefresh) {
        return null
    }

    if (!Authenticated) {
        return <Navigate to={redirectPath} replace={true} />
    }
    return <Outlet />
}

```

Para la conexión con backend, primero comenzamos ordenando los endpoints como se muestra en el siguiente código.

```

export const Services = {
    auth: {
        signUp: 'createUser',
    },
    users: {

```

```

    update: 'updateUser',
    getUserById: 'getUserById',
    getAll: 'getUserOrDriver?type=user',
  },
  table: {
    find: 'findInTable',
  },
} as const

```

En el siguiente fragmento de código se muestra como hacemos el llamado a cada endpoint relacionado con los usuarios.

```

export const signUpUser = async ({ email, password, fullName, username }:
SignUp) => {
  try {
    const response = await Moralis.Cloud.run(Services.auth.signUp, {
      objectData: {
        email,
        password,
        username,
        fullName,
      },
    })

    if (response.status === 'error') {
      return {
        error: 'Ha ocurrido un error',
      }
    }

    return {
      success: 'Registro exitoso',
    }
  } catch (error) {
    console.error(error)
    return {
      error: 'Ha ocurrido un error',
    }
  }
}

```

En la siguiente imagen se muestra cómo gestionamos estados y acciones con Zustand.

```
import type { User } from '@types'

export type UserState = {
  User: User | null
  Authenticated: boolean
  LoadingUser: boolean
  LoadingRefresh: boolean
  LoadingUserEdit: boolean
}

export type UserActions = {
  setUser: (user: User) => void
  setAuthenticated: (value: boolean) => void
  setLoadingUser: (value: boolean) => void
  setLoadingRefresh: (value: boolean) => void
  setLoadingUserEdit: (value: boolean) => void
}

// biome-ignore lint/suspicious/noExplicitAny: <explanation>
export const useUserStore = (set: any) => ({
  User: null,
  ⚠Authenticated: false,
  LoadingUser: false,
  LoadingRefresh: true,
  UserById: null,
  LoadingUserEdit: false,

  setLoadingUser: (value: boolean) => set({ LoadingUser: value }),
  setUser: (user: User) => set(() => ({ User: user })),
  setAuthenticated: (value: boolean) => set({ Authenticated: value }),
  // biome-ignore lint/suspicious/noExplicitAny: <explanation>
  setUserById: (user: any) => set(() => ({ UserById: user })),
  setLoadingUserEdit: (value: boolean) => set({ LoadingUserEdit: value }),
  setLoadingRefresh: (value: boolean) => set({ LoadingRefresh: value }),
})
```

[Ver imagen](#)

Gracias a la adopción de la arquitectura Screaming Architecture en el lado del Frontend se garantiza la claridad, mantenibilidad y escalabilidad del código. Esta arquitectura asegura que la estructura de nuestro código grite su intención, haciendo que cada proyecto sea más fácil de entender y trabajar para cualquier desarrollador.

Siendo así, a través de este enlace pueden visualizar la imágenes del proyecto

[Visitando este enlace podrás tener acceso a todas las capturas de código del proyecto](#)

Backend

1. Lógica del servidor Parse Server y Express:

Parse Server: Es un servidor de aplicaciones de código abierto que simplifica el desarrollo del backend para aplicaciones móviles, web y otras plataformas. Funciona como un Backend como Servicio (BaaS), encargándose de la lógica y gestión de datos del lado del servidor, lo que permite concentrarse en la interfaz y experiencia del usuario. Su flexibilidad lo hace compatible con cualquier tipo de cliente, incluyendo aplicaciones móviles nativas, sitios web, videojuegos desarrollados con motores como Unity, y otras plataformas digitales.

Características principales de Parse Server:

- **Base de datos en tiempo real:** Almacena y sincroniza datos en tiempo real, lo que permite que las aplicaciones se actualicen instantáneamente para todos los usuarios.
- **Autenticación de usuarios:** Proporciona un sistema seguro para autenticar y autorizar usuarios.
- **Almacenamiento de archivos:** Permite almacenar y administrar archivos de forma segura.
- **Notificaciones push:** Envía notificaciones push a los usuarios en sus dispositivos.
- **Código en la nube:** Ejecuta código JavaScript en el servidor para ampliar la funcionalidad de la aplicación.
- **Escalable:** Puede manejar un gran número de usuarios y solicitudes.
- **De código abierto:** Permite personalizar y adaptar el servidor a las necesidades específicas.

Beneficios de usar Parse:

- **Facilidad de uso:** Ofrece una API simple y fácil de usar, lo que la hace accesible para desarrolladores de todos los niveles.
- **Flexibilidad:** Es altamente personalizable y se puede adaptar a una amplia gama de aplicaciones.
- **Escalabilidad:** Puede manejar un gran número de usuarios y solicitudes, lo que la convierte en una buena opción para aplicaciones que esperan crecer.
- **Express:** Un marco web minimalista y flexible para Node.js que facilita la construcción de aplicaciones web y APIs. Es conocido por su simplicidad y velocidad, permitiendo el desarrollo de servidores HTTP.

2. Bases de datos y almacenamiento en el backend

MongoDB: Es una base de datos NoSQL de código abierto, orientada a documentos y distribuida. En lugar de almacenar datos en tablas como las bases de datos relacionales tradicionales, MongoDB almacena datos en documentos JSON flexibles. Esto la hace ideal para almacenar y consultar datos no estructurados o semiestructurados, como datos de redes sociales, datos de sensores y datos de IoT.

Características principales de MongoDB:

- **Modelo de datos flexible:** Almacena datos en documentos JSON, lo que permite estructuras de datos no rígidas y dinámicas.
- **Esquema flexible:** No requiere un esquema predefinido, lo que permite agregar, eliminar y modificar campos de documentos fácilmente.
- **Consulta potente:** Soporta un lenguaje de consulta rico y flexible, que incluye consultas ad hoc, agregaciones y geoespaciales.
- **Escalabilidad horizontal:** Se puede distribuir en múltiples servidores para manejar grandes volúmenes de datos y tráfico.
- **Alto rendimiento:** Ofrece un rendimiento rápido y escalable para aplicaciones exigentes.
- **De código abierto:** Libre de usar y modificar, lo que permite la personalización y adaptación a necesidades específicas.

A continuación se muestra de manera referencial la vista de la base de datos de mongodb

objectid	emailVerified	ACL	updatedAt	authData	username	createdAt	password	email
YdABZ80HfJ	(undefined)	Public Read, YdABZ...	16 May 2024 at 22:...	(undefined)	franciscoaugust	16 May 2024 at 22:...	(hidden)	franciscoaugustfa...
Tepqk5A3JC	(undefined)	Public Read, Tepqk...	16 May 2024 at 22:...	(undefined)	golfredopf	16 May 2024 at 22:...	(hidden)	golfredo.pf@gmail...
XBkIPp0yc	(undefined)	Public Read, XBkIL...	16 May 2024 at 22:...	(undefined)	gustavohernandez	16 May 2024 at 22:...	(hidden)	hernandez29aigmail...
FlsZLueuZK	(undefined)	Public Read, FlsZL...	16 May 2024 at 22:...	(undefined)	caructb	16 May 2024 at 22:...	(hidden)	caructb@gmail.com
R00qv178yB	(undefined)	Public Read, R00qv...	16 May 2024 at 22:...	(undefined)	augustbenito	16 May 2024 at 22:...	(hidden)	augustbenitogroup@...
cCET2Qk3y0	(undefined)	Public Read, cCETZ...	16 May 2024 at 22:...	(undefined)	zuccadev	16 May 2024 at 22:...	(hidden)	zuccadev@gmail.com

con el visor de parser-server

[Ver imagen de la base de datos](#)

Beneficios de Mongo DB:

- **Flexibilidad:** Es ideal para datos no estructurados o semiestructurados que no encajan bien en un modelo relacional.
- **Escalabilidad:** Puede manejar grandes volúmenes de datos y tráfico de forma eficiente.
- **Rendimiento:** Ofrece un rendimiento rápido y consistente para aplicaciones exigentes.
- **Facilidad de uso:** El modelo de datos flexible y el lenguaje de consulta intuitivo la hacen fácil de usar y aprender.
- **Comunidad grande y activa:** Cuenta con una comunidad grande y activa de desarrolladores que brindan soporte y comparten recursos.

Base de datos

Modelo de Datos:

En el caso de MongoDB y Parse Server, se utiliza un modelo de datos basado en documentos JSON. MongoDB es una base de datos NoSQL que almacena datos en documentos BSON (Binary JSON), y Parse Server utiliza MongoDB como su almacenamiento de datos predeterminado.

En este modelo, la información se organiza en documentos que son similares a objetos JSON, y estos documentos se almacenan en colecciones en lugar de tablas. Cada documento puede tener una estructura flexible, lo que significa que diferentes documentos en una misma colección pueden tener campos distintos.

1. Diagrama de Datos y Esquema de la base de datos

Diagramas de documento y de esquema de datos

#054 PLATAFORMA INTELIGENTE PARA RECRUITING ÓPTIMO DE PERSONAL FREELANCE

Diagrama de esquema de datos

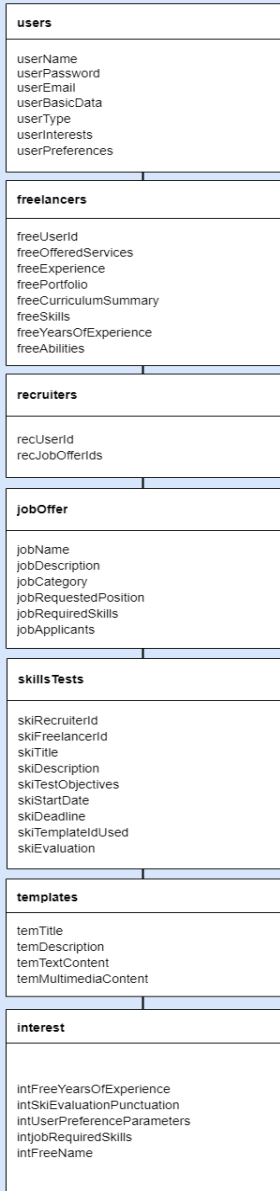


Diagrama de documento

```
Collection users
[-"userName": "Nombre de usuario",
-"userPassword": "Contraseña",
-"userEmail": "Correo electrónico",
-"userBasicData": {
  "firstNames": "Nombres",
  "lastNames": "Apellidos",
  "documentId": "Rif / nit / documento de identidad",
  "gender": "Género",
  "phoneNumber": "Número de teléfono"
},
-"userType": "Tipo de usuario",
-"userInterests": "Intereses",
-"userPreferences": {
  "settings": "Configuración",
  "parameters": "Parámetros"
}]

Collection freelancers
[-"freeUserId": "Id de usuario",
-"freeOfferedServices": "Servicios ofrecidos",
-"freeExperience": "Experiencia",
-"freePortfolio": "Portafolio",
-"freeCurriculumSummary": "Resumen curricular",
-"freeSkills": "Aptitudes",
-"freeYearsOfExperience": "Años de experiencia",
-"freeAbilities": "Habilidades técnicas y blandas que posee el freelancer"
-"freeName": "nombre del freelancer"]

Collection recruiters
[-"recUserId": "Id de usuario",
-"recJobOffers": "Id ofertas de empleo"]

Collection jobOffer
[-"jobName": "Nombre de empleo",
-"jobDescription": "Descripción",
-"jobCategory": "Categoría",
-"jobRequestedPosition": "Cargo solicitado",
-"jobRequiredSkills": "Aptitudes requeridas",
-"jobApplicants": {
  "freelancersId": "Id freelancer (Datos de la colección freelancers)",
  "status": "Estado del postulante",
  "skillsTestScore": "Puntuación en la prueba"
}]

Collection skillsTests
[-"skiRecruiterId": "Id reclutador",
-"skiFreelancerId": "Id freelancer",
-"skiTitle": "Título",
-"skiDescription": "Descripción",
-"skiTestObjectives": "Objetivos de la prueba",
-"skiStartDate": "Fecha de inicio",
-"skiDeadline": "Fecha límite",
-"skiTemplateIdUsed": "Id plantilla usada",
-"skiEvaluation": {
  "punctuation": "Puntuación de la evaluación",
  "correctAns": "respuesta correcta",
  "wrongAns": "respuesta incorrecta",
  "status": "Estado de la evaluación del postulante (aprobado o no)"
}]

Collection templates
[-"temTitle": "Título",
-"temDescription": "Descripción",
-"temTextContent": "Contenido textual",
-"temMultimediaContent": "Contenido multimedia",
-"temObjectives": "Objetivos",
-"temCategory": "Categoría"]

Collection interest
[-"intFreeYearsOfExperience": "Número de años de experiencia",
-"intSkiEvaluationPunctuation": "Puntuación de la prueba técnica",
-"intUserPreferenceParameters": "Parámetros de preferencias para un aspirante en la parte técnica y blanda",
-"intJobRequiredSkills": "Aptitudes requeridas de la oferta",
-"intFreeName": "nombre del freelancer"]
```

Datos comunes en todas las tablas

- Objectid
- ACL
- updateAt
- createAt
- authData

Datos comunes de la tabla user

- name
- email
- pass

[Ver diagrama](#)

Esquema de datos en formato json.

[Ver JSON](#)

2. Datos clave-valor y Optimización Redis

Caching de Consultas Frecuentes:

Parse Server identifica consultas o resultados de consultas que son frecuentemente utilizados, pero no cambian con frecuencia.

Estos resultados se almacenan en Redis como pares clave-valor, donde la clave puede ser un identificador único asociado a la consulta y el valor es el resultado de la consulta.

Configuración de Parse Server:

Parse Server se configura para utilizar Redis como su sistema de almacenamiento en caché. Esto se hace mediante la configuración del parámetro `cacheAdapter` durante la inicialización.

Uso en Consultas:

Antes de realizar consultas a la base de datos principal, Parse Server verifica si los datos requeridos están presentes en la caché de Redis.

Si los datos están en la caché, se evita la consulta a la base de datos principal, mejorando significativamente el rendimiento.

Manejo de Expiración:

Parse Server configura tiempos de expiración para los datos almacenados en caché en Redis.

Esto asegura que los datos no estén obsoletos y se vuelvan a consultar de la base de datos principal periódicamente.

Reducción de Carga en la Base de Datos Principal:

Al almacenar en caché datos frecuentemente utilizados, Parse Server reduce la carga en la base de datos principal (por ejemplo, MongoDB).

Esto mejora el rendimiento general de la aplicación al disminuir la necesidad de realizar consultas intensivas en la base de datos.

Al implementar este enfoque de almacenamiento en caché con Redis, Parse Server logra una optimización significativa en términos de rendimiento al reducir la carga en la base de datos principal y al acelerar el acceso a datos frecuentemente utilizados.

3. Políticas de Seguridad

En Parse Server con MongoDB, las políticas de seguridad están relacionadas con cómo se controla el acceso a la base de datos MongoDB y cómo se aseguran los datos almacenados. políticas de seguridad en este entorno:

Autenticación en MongoDB:

MongoDB utiliza un sistema de autenticación basado en roles y usuarios. Parse Server debe autenticarse correctamente con MongoDB para acceder a los datos.

Las credenciales de acceso (nombre de usuario y contraseña) deben ser seguras y únicas para garantizar la autenticación segura entre Parse Server y MongoDB.

Autorización y Roles:

MongoDB permite la asignación de roles y permisos a los usuarios. Parse Server puede definir roles específicos para acceder a ciertos recursos o colecciones en MongoDB.

Es posible restringir las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) basadas en roles para garantizar que los usuarios tengan solo los permisos necesarios.

Configuración de Parse Server:

La conexión entre Parse Server y MongoDB se establece de manera segura, utilizando conexiones cifradas (SSL/TLS) para proteger la transferencia de datos, en producción.

Políticas de Seguridad de Parse Server:

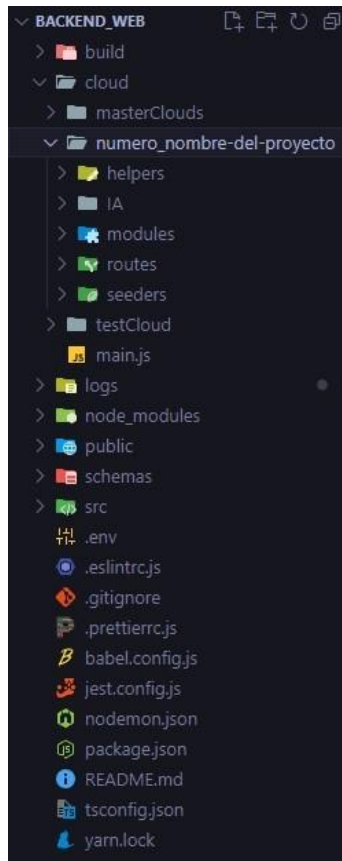
Parse Server implementa políticas de seguridad a nivel de aplicación, permitiendo definir quién tiene acceso a los datos y qué operaciones pueden realizar.

Se pueden configurar ACLs (Listas de Control de Acceso) en los objetos Parse para definir quién puede acceder, leer, escribir o eliminar esos objetos.

Descripción de la arquitectura del backend:

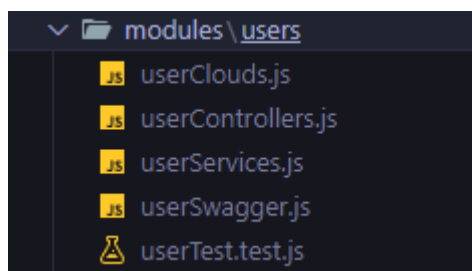
Se utiliza en esta versión una estructura hexagonal en donde tenemos el controlador que es a donde se recibe la petición del frontal , luego la capa de datos en donde se encuentra la lógica del negocio para después irnos a la capa de database en donde se almacena el objeto.

Arquitectura de Parser-server



[ver imagen](#)

se usó una arquitectura hexagonal, en donde dentro de la carpeta modules está todo el funcionamiento de un módulo el cual se describe a continuación tomando como ejemplo el módulo de usuario:



[ver imagen](#)

En el userClouds se centralizan todos los endpoints que va utilizar la aplicación:

```
import { registerTag } from '../../../src/swagger/swaggerTags';
import {
  createUserNewControllers,
  deleteUserControllers,
  getAllUsersControllers,
  getUserByIdControllers,
  updateUserControllers
} from
```

```

'./userControllers';

registerTag('User', 'Todas las rutas relacionadas al usuario');

Parse.Cloud.define('getAllUsers', getAllUsersControllers(Parse));
Parse.Cloud.define('getUserById',      getUserByIdControllers(Parse),      {
requireUser: true });
Parse.Cloud.define('createUser', createUserNewControllers(Parse));
Parse.Cloud.define('deleteUser', deleteUserControllers(Parse), { requireUser:
true });
Parse.Cloud.define('updateUser', updateUserControllers(Parse), { requireUser:
true });

    };
}

```

En el userController se recibe lo que envía el frontal ya procesado y validado por los protocolos de parser-server, este es un segmento del código para crear un usuario,

```

export function createUserNewControllers(Parse) {
  return async (request) => {
    try {
      const { objectData } = request.params;
      const user = await createNewUserServices(objectData);

      return {
        status: 'success',
        result: true,
        user,
      };
    } catch (error) {
      console.error(`Error   code:   ${error.code},   Error   message:
${error.message}`);
      return {

```

```

        status: 'error',
        result: false,
        errorDetails: {
            code: error.code || Parse.Error.INTERNAL_SERVER_ERROR,
            message: error.message,
        },
    };
}
};
}

```

En `userService` se encuentra la lógica del negocio, es en este punto en donde se instancia la base de datos y se hacen las diferentes funcionalidades que comprenden los proyectos, una vez procesados los datos se procede a guardarlos en la colección que les corresponde:

```

export async function createNewUserServices(objectData) {

    try {

        if (!objectData) {

            throw new Parse.Error(Parse.Error.OBJECT_NOT_FOUND, 'objectData is missing.');
        }

        const User = Parse.Object.extend('User');

        const user = new User();

        for (const key in objectData) {

            if (objectData.hasOwnProperty(key)) {

                user.set(key, objectData[key]);

            }

        }

        await user.save();

        return user;
    }
}

```



```
} catch (error) {  
  
    throw error;  
  
}  
  
}
```

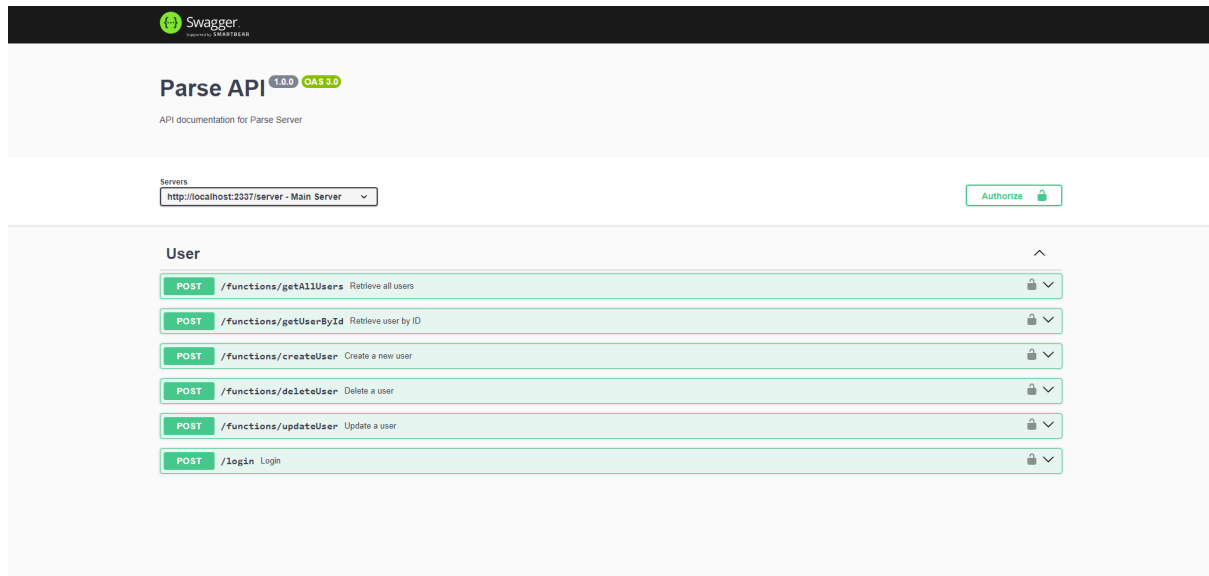
En userSwagger se crea toda la documentación necesaria, esta funcionalidad es usada por el frontal para saber cómo funcionan los diferentes endpoints que comprenden la aplicación y probar el funcionamiento de cada uno de ellos.

Mostramos el código:

```
export const getUserByIdSwagger = {  
  '/functions/getUserById': {  
    post: {  
      tags: ['User'],  
      summary: 'Retrieve user by ID',  
      security: [  
        { sessionAuth: [] },  
        { appIdAuth: [] }  
      ],  
      requestBody: {  
        description: 'ID of the user',  
        required: true,  
        content: {  
          'application/json': {  
            schema: {  
              type: 'object',  
              properties: {  
                userId: {  
                  type: 'string',  
                  description: 'ID of the user',  
                  example: 'WYSpv8cCyA'  
                }  
              },  
            },  
            required: ['userId']  
          }  
        }  
      }  
    }  
  }  
}
```

```
    }  
  },  
  responses: {  
    200: {  
      description: 'Successful response',  
      content: {  
        'application/json': {  
          schema: {  
            type: 'object'  
          }  
        }  
      }  
    }  
  }  
}  
};
```

y el como se ve en un navegador



[ver imagen](#)

y el último archivo que está en la carpeta de user es la de testing que es en donde esta la configuración de todas las pruebas que comprenden el módulo en cuestión.

```
describe('Update User', () => {
  let updateUserResponse;
  beforeAll(async () => {
    updateUserResponse = await request(app)
      .post('/server/functions/updateUser')
      .set('X-Parse-Application-Id', `${applicationId}`)
      .set('X-Parse-REST-API-Key', masterKey)
      .set('X-Parse-Session-Token', sessionToken)
      .send({
        userId: createUserResponse.body.result.user.objectId,
        objectData: {
          username: 'example123',
        },
      });
  });

  it('The response status is "success"', async () => {
    expect(updateUserResponse.body.result.status).toBe('success');
  });

  it('The response should contain a user object', async () => {
    expect(updateUserResponse.body.result).toHaveProperty('user');
  });
});
```

```

});

it('The response contains a result object', async () => {
  expect(updateUserResponse.body.result.result).toBe(true);
});
});

```

En la carpeta de seeders se tiene la configuración del llenado de la base de datos con los datos scrapeados por parte de IA.

```

import { loadAllData } from '../helpers/loadAllData';

export function seedersMain(Parse) {
  return async (request) => {
    try {
      await loadAllData();

      return {
        status: 'success',
        result: true,
        message: 'Seeding successfully completed.',
      };
    } catch (error) {
      console.error(`Código de error: ${error.code}, Mensaje de error: ${error.message}`);
      return {
        status: 'error',
        result: false,
        errorDetails: {
          code: error.code || Parse.Error.INTERNAL_SERVER_ERROR,
          message: error.message,
        },
      };
    }
  };
}

```

luego en la carpeta Schema se tiene la configuración de los acl y la configuración de la seguridad en donde se coloca que campo es requerido , que tipo de roll tiene acceso y que

puede hacer , así como también la declaración de las referencias de una colección a otra, se toma de ejemplo el schema de usuario.

```
export const User = {
  className: "_User",
  fields: {
    emailVerified: { type: "Boolean", defaultValue: false },
    phone: { type: "String", required: true },
    user_type: { type: "String", required: true },
    authData : { type: "Object" },
    username: { type: "String", required: true },
    user_birthDate: { type: "Date" },
    fullname: { type: "String", required: true },
    password: { type: "String" },
    email: { type: "String", required: true },
    accounts: { type: "Array" },
    ethAddress : { type: 'String' },
    nemotecnic: { type: 'String' },
    privateKey : { type: 'String' },
    validatePing: { type: "Boolean" },
  },
  /* indexes: {
    tagsIndex: { tags: 1 },
    // The special prefix _p_ is used to create indexes on pointer fields
    cityPointerIndex: { _p_city: 1 },
    tagAndCityIndex: { _p_city: 1, tags: 1 },
  }, */
  classLevelPermissions: {
    find: { requiresAuthentication: true },
    // *count: { "role:Admin": true },
    count: { "*": true },
    get: { "*": true },
    addField: { 'role:admin': true },
    update: { 'role:admin': true },
    create: { '*': true },
    delete: { "role:admin": true },
    protectedFields: {
      // These fields will be protected from all other users. AuthData and
      password are already protected by default
      "*": ["emailVerified", "authData", "password", "privateKey",
        "ethAddress", "nemotecnic", "privateKey", "validatePing"],
    }
  }
}
```

```
    },  
  },  
};
```

y esta funcionalidad es llamada en el archivo de parserServer.ts en donde se debe bajar el servidor y luego subirlo para que tome estos cambios

```
import { User } from '../schemas/schema';  
  
// Define schemas of Parse Server  
schema: {  
  definitions: [User], //? aqui van las tablas a generar con su seguridad  
  // If set to true, the Parse Server API for schema changes is disabled  
and schema  
  // changes are only possible by redeploying Parse Server with a new schema  
definition  
  lockSchemas: true,  
  // If set to true, Parse Server will automatically delete non-defined  
classes from  
  // the database; internal classes like User or Role are never deleted.  
  strict: true,  
  // If set to true, a field type change will cause the field including its  
data to be  
  // deleted from the database, and then a new field to be created with the  
new type  
  recreateModifiedFields: false,  
  // If set to true, Parse Server will automatically delete non-defined  
class fields;  
  // internal fields in classes like User or Role are never deleted.  
  deleteExtraFields: true,  
},
```

Siendo así, a través de este enlace se pueden visualizar las imágenes del proyecto

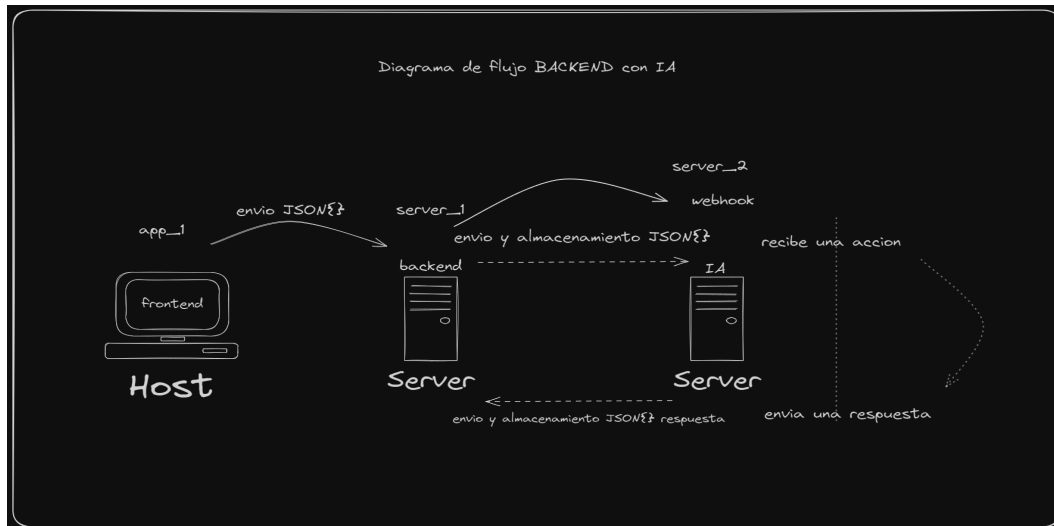
[Visitando este enlace podrás tener acceso a todas las capturas de código del proyecto](#)

4. Integración de IA mediante microservicios:

En este contexto, la integración de IA a través de microservicios implica dividir la lógica de inteligencia artificial en servicios independientes que se comunican a través de webhooks.

Este enfoque permite una arquitectura más modular y escalable, donde la IA puede ser actualizada y escalada de manera independiente al resto de la aplicación.

Diagrama de Lógica de comunicación con IA:



5. Despliegue de servicios cloud computing

Sinergy Elastic Servers: Una opción de despliegue en la nube ofrecida por Sinergy que proporciona servidores virtuales escalables. Permite adaptarse a las necesidades específicas del proyecto, con la capacidad de escalar automáticamente para manejar variaciones en la demanda.

Sinergy Apps Service: Un servicio de plataforma como servicio (PaaS) ofrecido por Sinergy que simplifica el proceso de implementación y gestión de aplicaciones. Permite a los desarrolladores centrarse en el código sin preocuparse por la infraestructura subyacente.

6. Seguridad y privacidad con Express y Parse-Server:

Ambas tecnologías ofrecen herramientas y funcionalidades para implementar medidas de seguridad, como autenticación y autorización. El uso de HTTPS es esencial para proteger las comunicaciones.

Inteligencia Artificial

A continuación se proporciona una descripción detallada del proyecto en términos de inteligencia artificial (IA) centrado en un modelo de recomendaciones. El proyecto combina estratégicamente datos scrapeados y datos simulados para construir un conjunto de datos completo y enriquecido que alimenta tanto al chatbot como al modelo de recomendaciones.

Origen y Naturaleza de los Datos

El conjunto de datos utilizado en este proyecto es el resultado de una combinación estratégica entre datos scrapeados y datos simulados. Esta fusión se llevó a cabo con el propósito de construir un dataset completo y enriquecido, capaz de abastecer al modelo de recomendaciones, asegurando así la diversidad y calidad de la información necesaria.

Datos Scrapeados:

Se obtuvieron mediante técnicas de web scraping de los siguientes sitios:

- [Visitar documento](#)

Datos Simulados:

Adicionalmente, se incorporaron datos ficticios simulados para completar las posibles brechas en la información y para proporcionar un conjunto de entrenamiento más robusto al modelo. Estos datos simulados se generaron siguiendo patrones realistas y escenarios que podrían surgir en el contexto del sistema de recomendación.

Generación de datos simulados con Faker para proyectos de IA

El proceso de generación de datos simulados para un proyecto de IA utilizando el módulo Faker.

- Definición de parámetros: Se definen los parámetros necesarios para la generación de datos, como el número de registros, el tipo de datos y los rangos de valores.
- Selección de campos: Se seleccionan los campos de la estructura de datos que se quieren generar.
- Generación de valores: Se utilizan las funciones de Faker para generar valores aleatorios para cada campo.
- Validación de datos: Se valida la calidad de los datos generados para garantizar que sean consistentes y representativos de los datos reales.
- Almacenamiento de datos: Se almacenan los datos generados en un formato adecuado, en este caso JSON.

Objetivos:

El objetivo principal de este proceso es generar datos simulados de alta calidad que puedan utilizarse para entrenar y evaluar modelos de IA, especialmente en situaciones donde no se dispone de datos reales suficientes o de calidad.

```
20     def getElementsByClass(self):
21         try:
22             self.driver.get(self.url)
23             time.sleep(10) # Esperar a que la página cargue
24             results = {}
25             for attr, class_name in self.field_classes.items():
26                 elements = self.driver.find_elements(By.CLASS_NAME, class_name)
27                 time.sleep(1) # Ajustar el tiempo de espera según sea necesario
28                 print(f'Se encontraron {len(elements)} elementos para {attr}')
29                 attr_values = [element.text for element in elements]
30                 results[attr] = attr_values
31
32             df = self.create_df(results)
33             self.obtainJsonOrExcelOrCsv(df)
34         except NameError as e:
35             print(f'Se produjo un error: {str(e)}')
36         finally:
37             self.driver.quit()
```

[Ver imagen](#)

```

1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.chrome.options import Options
4 import time
5 import pandas as pd
6 import json
7
8 class WebScraper:
9     def __init__(self, config_path):
10         with open(config_path, 'r') as file:
11             self.config = json.load(file)
12
13         options = Options()
14         options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36")
15         self.driver = webdriver.Chrome(options=options)
16         self.url = self.config["url"]
17         self.field_classes = self.config["field_classes"]
18         self.output_config = self.config["output"]

```

[Ver imagen](#)

```

66 def main():
67     scraper = WebScraper('config.json')
68     scraper.getElementsByClass()
69
70 if __name__ == "__main__":
71     main()
72

```

[Ver imagen](#)

```

1 {
2     "url": "nombre_url",
3     "field_classes": {
4         "key1": "className",
5         "key2": "className",
6         "key3": "className"
7     },
8     "output": {
9         "file_name": "fileName",
10        "extension": "json"
11    }
12 }

```

[Ver imagen](#)

Estructura de datos:

La estructura de datos del proyecto plataforma inteligente para el reclutamiento óptimo de personal freelance tiene los siguientes campos:

- **Nombre:** Cadena de texto aleatoria.
- **Apellido:** Cadena de texto aleatoria.
- **Edad:** Número entero aleatorio entre 18 y 65.
- **Correo electrónico:** Cadena de texto aleatoria con formato de correo electrónico.
- **Dirección:** Cadena de texto aleatoria con formato de dirección.
- **Teléfono:** Cadena de texto aleatoria con formato de número de teléfono.

Configuración de Faker:

En el código de ejemplo, **Configuración de Faker:**

Se crea una instancia de Faker utilizando la clase `faker.Faker()`. Esta instancia permite acceder a las funciones de Faker para generar valores aleatorios para diferentes tipos de datos.

Se crea una instancia de Faker utilizando la clase `faker.Faker()`. Esta instancia permite acceder a las funciones de Faker para generar valores aleatorios para diferentes tipos de datos.

Código Configuración

```
1  {
2      "key1": "fakerFuncionNam",
3      "key2": "fakerFuncionNam",
4      "key3": "fakerFuncionNam",
5      "key4": "",
6      "key5": "random fieldNameOfAnotherColectio",
7  }
```

[Ver imagen](#)

```

1 from faker_module import FakerModule
2 import os
3 import json
4 import random
5
6 # Función para cargar la colección desde un archivo JSON
7 def cargar_coleccion(nombre_archiv ):
8     with open(nombre_archiv , 'r') as f:
9         return json.load(f)
10
11 # Función para exportar los datos falsos a un nuevo archivo JSON
12 def exportar_coleccio (coleccion, nombre_archiv ):
13     with open(nombre_archiv , 'w') as f:
14         json.dump(coleccion, f, indent=4, ensure_ascii =False)
15
16 # Cargar la configuración desde un archivo JSON
17 config = cargar_coleccion('./config.json')
18
19 # Crear una instancia del módulo Faker con la configuración cargada
20 faker_module = FakerModule(config)
21
22
23 # Cargar la colección desde un archivo JSON
24 nombre_archivo_origina = './042_interests.json'
25 coleccion = cargar_coleccion(nombre_archivo_origina)
26
27 # Generar datos falsos extendidos
28 datos_falso = faker_module.extend_coleccio (coleccion, 1000)
29 # Por ejemplo, generar 1000 registros adicionales
30
31 # Construir el nombre del archivo exportado
32 nombre_base, extension = os.path.splitext(nombre_archivo_origina)
33 nombre_archivo_exportad = f"{nombre_base}_mod{extension}"
34
35 # Exportar los datos falsos a un nuevo archivo JSON con el nombre modificado
36 exportar_coleccio (datos_falso , nombre_archivo_exportad)
37
38 print(f"Los datos falsos han sido generados y guardados en {nombre_archivo_exportad}.")

```

[Ver imagen](#)

Proceso de Limpieza y Preprocesamiento de Datos

Los datos recolectados fueron sometidos a un proceso exhaustivo de limpieza y preprocesamiento para garantizar la calidad y coherencia de la información. Se llevaron a cabo las siguientes actividades:

Limpieza de Datos

- **Manejo de Datos Faltantes:** Se identificaron y abordaron los valores nulos o faltantes de manera adecuada, ya sea mediante imputación o eliminación según el contexto.
- **Detección y Tratamiento de Outliers:** Se realizaron análisis estadísticos para identificar posibles valores atípicos que pudieran afectar la integridad de los resultados y se aplicaron estrategias para gestionarlos.

Preprocesamiento

- **Normalización y Escalado:** Las variables numéricas fueron normalizadas o escaladas según la necesidad del modelo, para garantizar una comparación adecuada entre diferentes magnitudes.
- **Transformación de Variables Categóricas:** Se realizaron conversiones y codificaciones de variables categóricas para integrarlas efectivamente en el modelo.

Código

```
1 # data_cleaning.py
2
3 import pandas as pd
4 import re
5
6 def clean_dataframe(df, columns_to_drop=None):
7     """
8     Realiza una limpieza general del DataFrame
9     e.
10     Parámetro
11     s: - df: pd.DataFrame - DataFrame a limpiar
12     r. - columns_to_drop: list (opcional) - Lista de nombres de columnas a eliminar
13     r.
14     Return
15     a: - pd.DataFrame - DataFrame limpio
16     o.
17     """
18     # Eliminar columnas específicas
19     if columns_to_drop is not None:
20         pdf = df.drop(columns=columns_to_drop, errors='ignore')
21     # Manejar valores faltantes llenando con la media para columnas numéricas
22     df.fillna(df.mean(numeric_only=True), inplace=True)
23     # Eliminar duplicados
24     df.drop_duplicates(inplace=True)
25     # Remover caracteres especiales de los nombres de columna
26     df.columns = re.sub(r'[\W\s]', '', col) for col in df.columns
27     # Remover caracteres especiales de los datos en cada celda
28     df = df.applymap(lambda x: re.sub(r'[\W\s]', '', str(x)) if isinstance(x, str) else x)
29
30     return df
31
32 # Para importar este módulo en otro archivo
33 # from data_cleaning import clean_dataframe
34
35 e
```

[Ver imagen](#)

El módulo `data_cleaning.py` contiene una función llamada `clean_dataframe` que se centra en mejorar la calidad de los datos para su uso en proyectos, especialmente en contextos donde se requiere un preprocesamiento riguroso, como el entrenamiento de modelos de machine learning. La limpieza de datos es una etapa crucial en cualquier flujo de trabajo de análisis de datos, ya que garantiza que los datos sean precisos, consistentes y estén libres de errores o elementos innecesarios que puedan afectar el rendimiento de los modelos.

La función `clean_dataframe` realiza varias tareas fundamentales para asegurar la calidad de los datos. Primero, permite la eliminación de columnas no deseadas mediante el argumento opcional `columns_to_drop`, que acepta una lista de nombres de columnas a eliminar. Esto es útil para eliminar características irrelevantes o redundantes que no contribuyen al análisis o al entrenamiento del modelo.

Luego, la función maneja los valores faltantes, rellenándolos con la media de las columnas numéricas. Esta estrategia de imputación es simple, pero efectiva para evitar problemas que pueden surgir al tener valores nulos, especialmente en algoritmos de aprendizaje automático que no manejan bien los datos incompletos. La media es una elección común, ya que minimiza el sesgo que puede introducirse al reemplazar los valores faltantes.

La eliminación de duplicados es otro paso esencial que se realiza, puesto que los registros duplicados pueden distorsionar los resultados del análisis y afectar negativamente el rendimiento del modelo al introducir redundancia no deseada. Además, la función también

limpia los nombres de las columnas y los datos de caracteres especiales, utilizando expresiones regulares. Esto es especialmente importante en situaciones donde los datos provienen de múltiples fuentes y pueden contener inconsistencias en el formato, lo que puede causar errores durante el procesamiento o el entrenamiento del modelo.

Elección del Algoritmo de Machine Learning

[Enlace a la carpeta de documentos](#)

Implementación del algoritmo

[Enlace a la carpeta de imágenes](#)

Pruebas y Ajustes Fines

[Enlace al documento](#)